SEMANA 15 — INTELIGENCIA ARTIFICIAL EN LA NUBE II

IA en la Nube II: GPU, Cuantización y Despliegue

Parte 1 de 5 — Aceleradores de Hardware y Arquitectura de GPUs

Ing. Rodolfo Cañas Cervantes
Universidad de la Costa (CUC) — Ingeniería de Sistemas
Semana 15 — Periodo 2026-2 — Barranquilla, Colombia

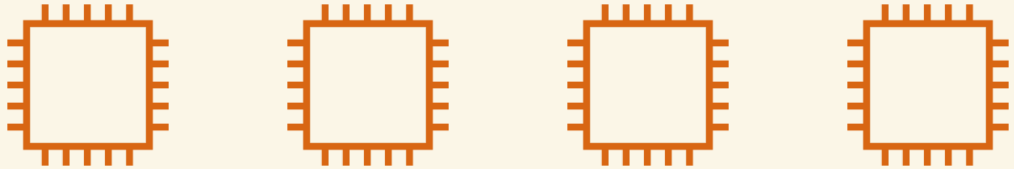
Parte 1 de 5 — Aceleradores de Hardware y Arquitectura de GPUs
IA = inteligencia artificial · TCO = costo total de propiedad

CPU frente a GPU para inferencia

Pocos núcleos complejos frente a miles simples

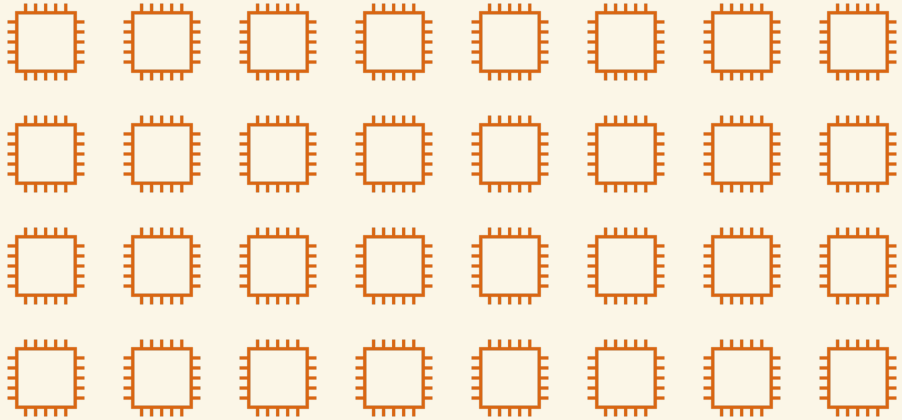
CPU

4 núcleos complejos



GPU

Miles de núcleos simples



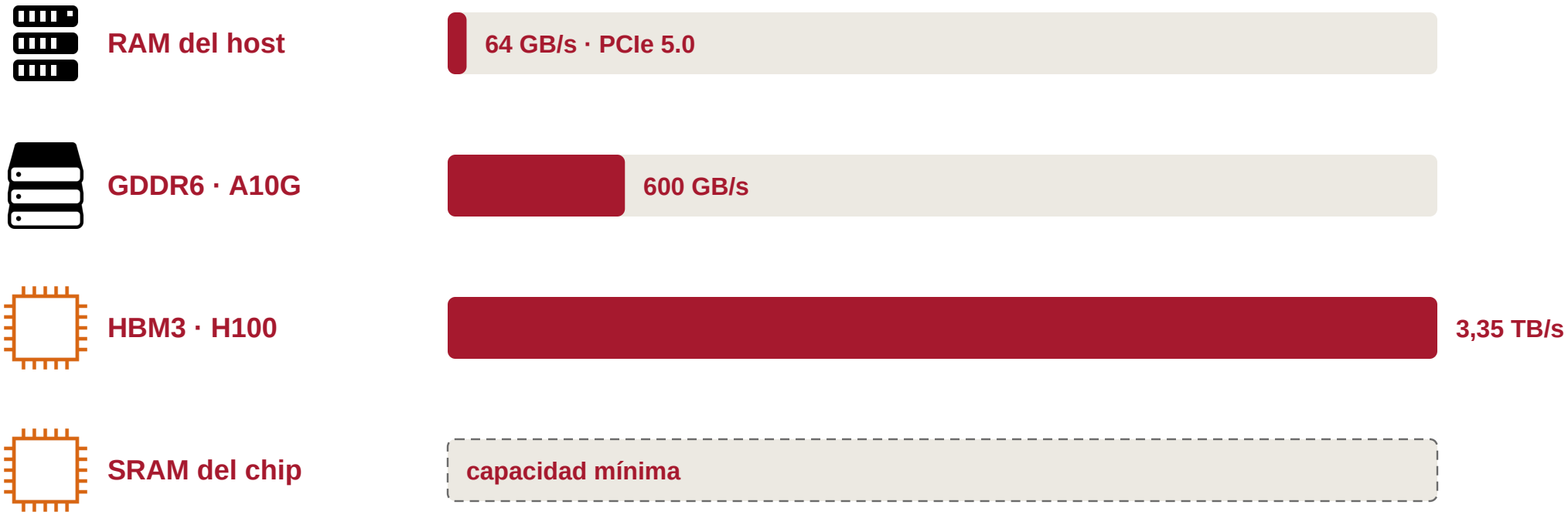
Una operación sobre miles de datos

CPU = unidad central de procesamiento · GPU = unidad de procesamiento gráfico

Fuente: NVIDIA, documentación de arquitectura de GPU (developer.nvidia.com)

Ancho de banda de la RAM a la SRAM

Dónde se mueven los pesos del modelo



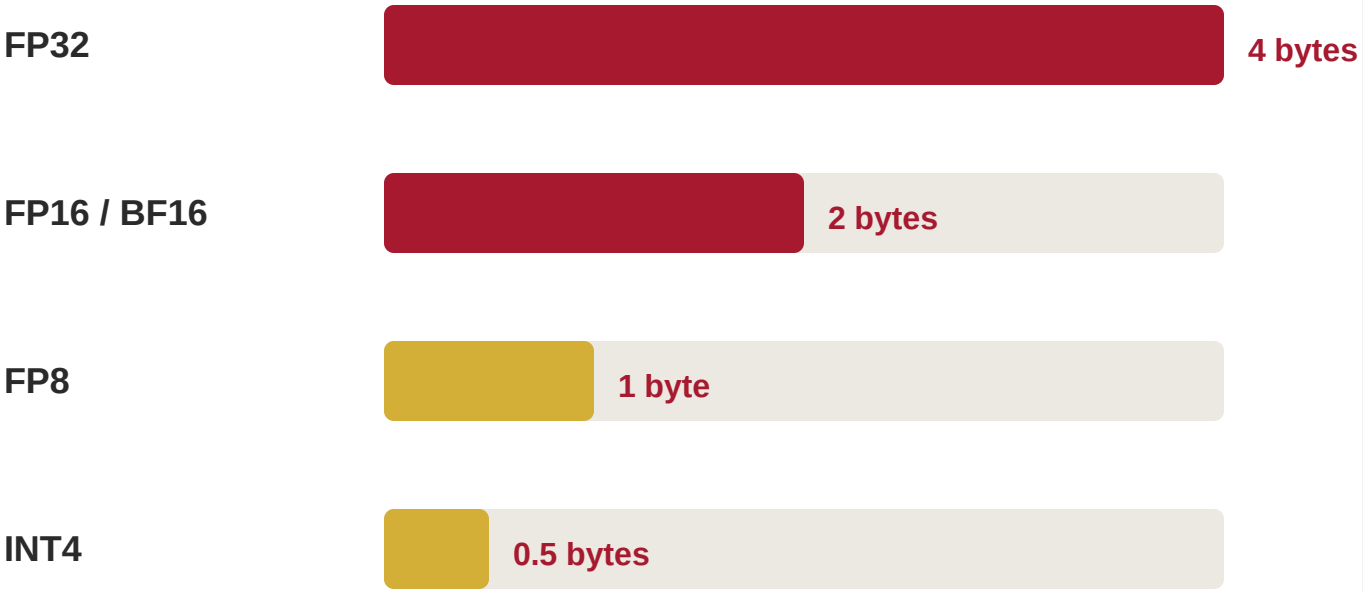
La inferencia está limitada por el ancho de banda

RAM = memoria de acceso aleatorio · GB = gigabyte · GDDR6 = memoria gráfica de sexta generación · HBM3 = memoria de alto ancho de banda, 3.ª generación · TB = terabyte · SRAM = memoria estática rápida

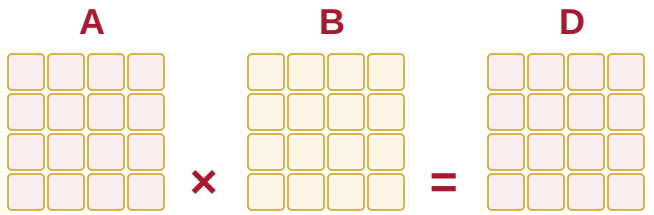
Fuente: NVIDIA, datasheets de A10 y H100 (nvidia.com); PCI-SIG, PCIe 5.0 (pcisig.com)

Menos bits por peso, más cálculo por byte

Bytes por parámetro y cálculo matricial



Tensor Core



Una operación por ciclo

Más cálculo, misma memoria

Menos bytes por peso: más parámetros por GB

FP32 = coma flotante de 32 bits · FP16 = coma flotante de 16 bits · BF16 = coma flotante de 16 bits de rango amplio · FP8 = coma flotante de 8 bits · INT4 = entero de 4 bits · GB = gigabyte

Fuente: NVIDIA, H100 Tensor Core GPU (nvidia.com/en-us/data-center/h100); bytes por formato, cálculo propio

NVLink (enlace directo) frente a PCIe (bus de tarjetas)

Enlace directo entre GPU frente al bus



Tarjetas H100 apiladas

Ancho de banda

PCIe Gen 5 — **64 GB/s**

NVLink 4 — **900 GB/s**

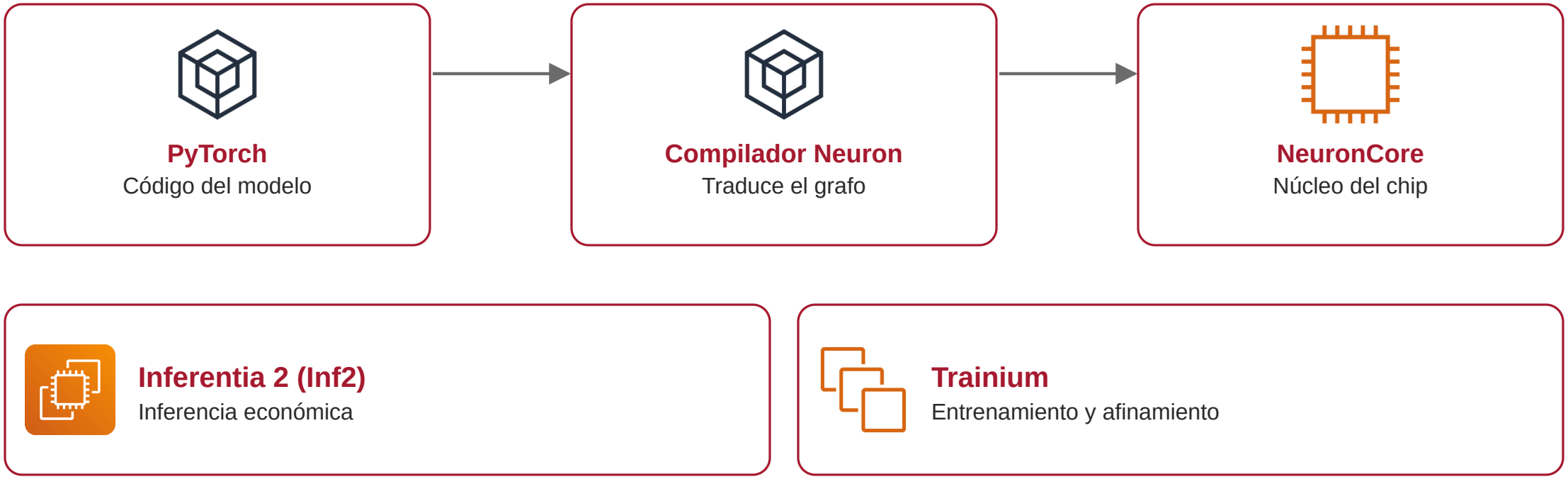
Sincronizar pesos sin pasar por la CPU

GB = gigabyte · CPU = unidad central de procesamiento · GPU = unidad de procesamiento gráfico

Fuente: NVIDIA, NVLink y NVSwitch (nvidia.com/en-us/data-center/nvlink) · Foto: Geekerwan, Wikimedia Commons (CC BY 3.0)

Inferentia y Trainium de AWS

Silicio propio que exige un compilador



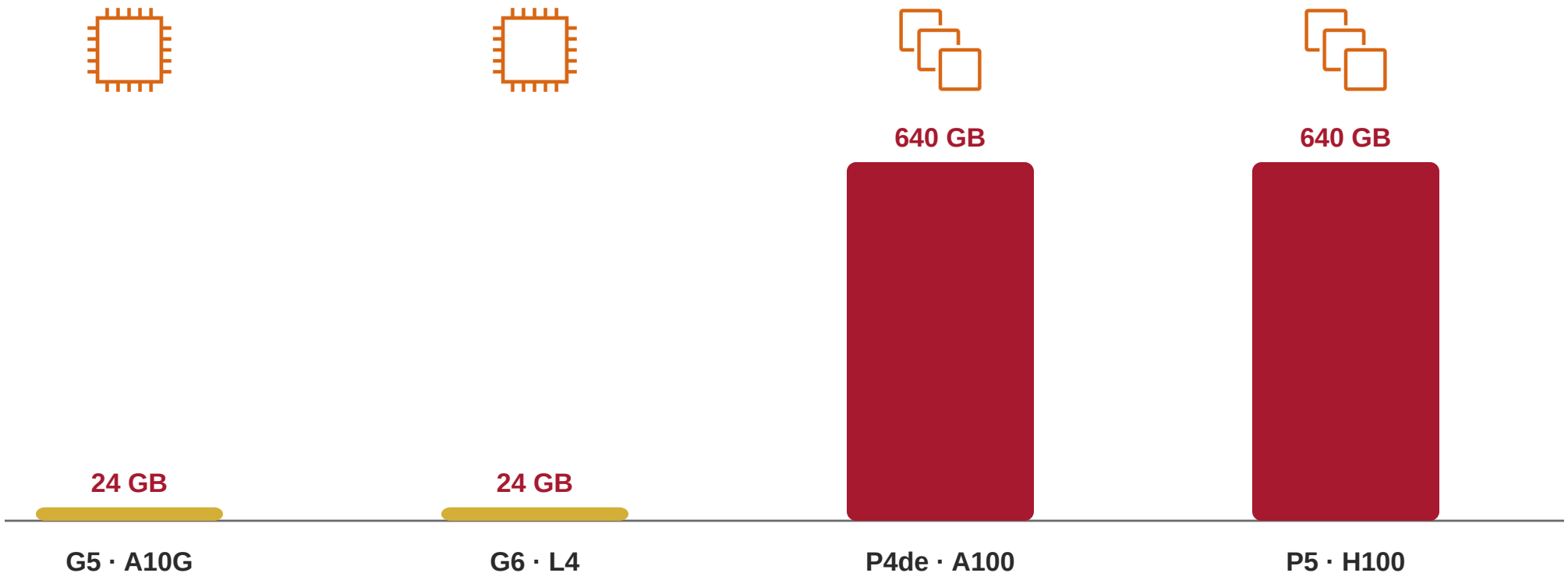
Peaje: compilación previa

AWS = Amazon Web Services

Fuente: AWS, Inferentia y Trainium (aws.amazon.com/machine-learning/inferentia; aws.amazon.com/machine-learning/trainium)

Familias de GPU en AWS EC2

Memoria de GPU por instancia (escala común)



La memoria de la GPU define la familia

GB = gigabyte · L4 = capa 4 (transporte) · GPU = unidad de procesamiento gráfico · AWS = Amazon Web Services · EC2 = cómputo elástico de Amazon

Fuente: aws.amazon.com/ec2/instance-types/ (G5, G6, P4de, P5); g5.xlarge $1,006/h, AWS Price List us-east-1 (consulta 2026-11-05)

EFA y RDMA frente a TCP/IP

Cómo viaja un tensor entre dos servidores



Switches de centro de datos

TCP/IP clásico

VRAM → RAM del host → SO (kernel) → NIC

EFA + RDMA

VRAM local → EFA → Red SRD → VRAM remota

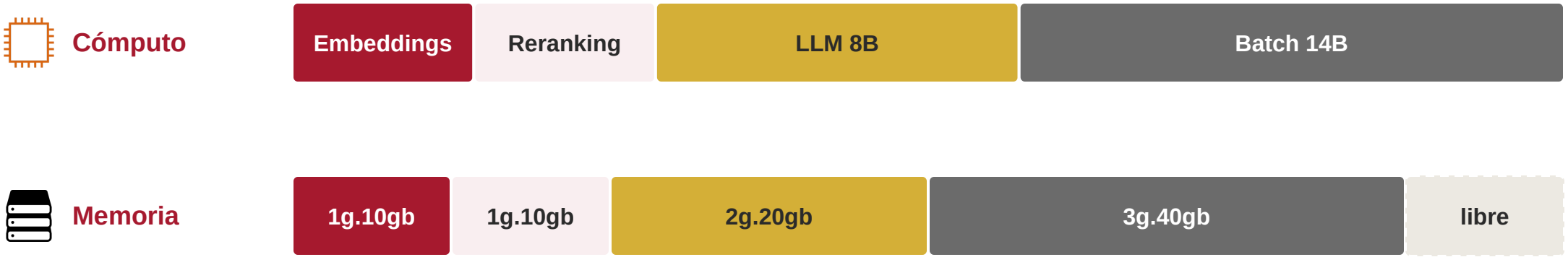
Saltar el sistema operativo reduce la latencia

TCP = protocolo de control de transmisión · IP = protocolo de internet · VRAM = memoria de la GPU · RAM = memoria de acceso aleatorio · SO = sistema operativo · NIC = tarjeta de red · EFA = adaptador de red elástico de Amazon Web Services · RDMA = acceso directo a memoria remota · SRD = entrega fiable multicamino de EFA

Fuente: AWS, Elastic Fabric Adapter (aws.amazon.com/hpc/efa) · Foto: switches, Dsimic, Wikimedia Commons (CC BY-SA 4.0)

NVIDIA Multi-Instance GPU (MIG)

Cuatro servicios en una sola GPU



Embeddings
10 GB

Reranking
10 GB

LLM 8B
20 GB

Batch 14B
40 GB

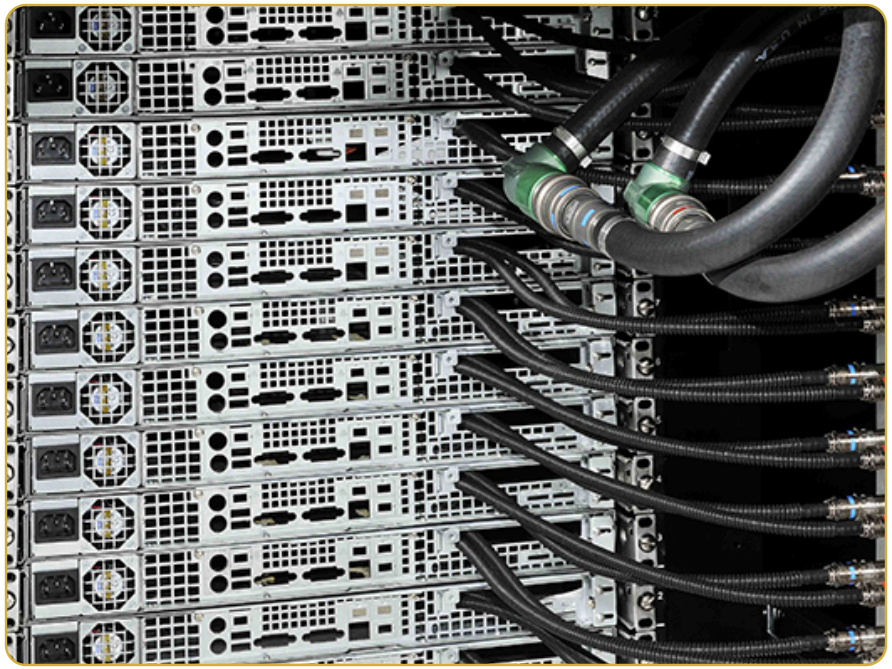
Aislamiento físico

LLM = modelo de lenguaje de gran tamaño · GB = gigabyte · GPU = unidad de procesamiento gráfico · NVIDIA = fabricante de GPU

Fuente: NVIDIA, Multi-Instance GPU User Guide (docs.nvidia.com/datacenter/tesla/mig-user-guide)

Potencia y refrigeración en centros de datos

Densidad de potencia y refrigeración



Rack refrigerado

Potencia por rack

Un servidor — **10,2 kW**

Cuatro servidores — **≈41 kW**

1,54 **PUE**

Uptime 2025

Líquido: menor PUE

PUE = efectividad energética (energía total entre la de los equipos)

Fuente: NVIDIA, DGX H100 (10,2 kW máx.); Uptime Institute, Global Data Center Survey 2025 (PUE 1,54) · Foto: CoolIT, Wikimedia Commons (CC BY-SA 3.0)

GPU passthrough en OpenStack

Una GPU dedicada a una máquina virtual



Una GPU, una VM

VFIO = controlador de paso directo de dispositivos · GPU = unidad de procesamiento gráfico · NVIDIA = fabricante de GPU · VM = máquina virtual

Fuente: OpenStack, PCI passthrough (docs.openstack.org/nova/latest/admin/pci-passthrough.html)

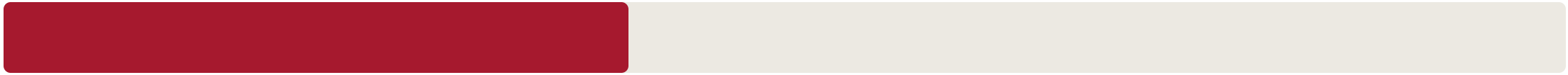
Rendimiento real frente al teórico: MFU

Del pico del fabricante a la capacidad real

Pico teórico



MFU real (ilustrativo)



MFU = FLOPS usados ÷ FLOPS pico

Dimensionar con el MFU medido, no con el pico

FLOPS = operaciones de coma flotante por segundo

Fuente: Chowdhery et al. (2022), PaLM, arXiv:2204.02311 (definición de MFU); cálculo propio

Monitoreo de GPU con NVIDIA DCGM

Del sensor de la tarjeta a la alerta



GPU + DCGM
Métricas de GPU

Prometheus
Raspado periódico

Grafana
Paneles en vivo

KEDA / HPA
Escala y drena

Un XID: aislar el nodo y reemplazarlo

GPU = unidad de procesamiento gráfico · DCGM = monitor de GPU de Nvidia · KEDA = autoescalado dirigido por eventos · HPA = autoescalado horizontal de pods · XID = código de error de GPU de Nvidia · NVIDIA = fabricante de GPU

Fuente: NVIDIA, Data Center GPU Manager (DCGM) y catálogo de errores XID (docs.nvidia.com/deploy/xid-errors)

CASO ILUSTRATIVO

Selección de hardware para inferencia

Pesos FP16 frente a la VRAM

Llama 8B

Pesos: 16 GB



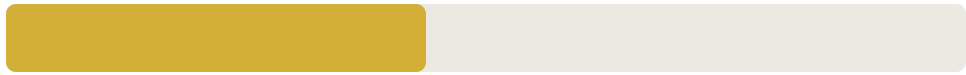
A10G: 24 GB



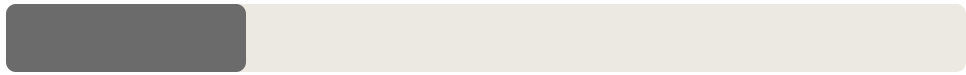
Cabe: 1 GPU

Llama 70B

Pesos: 140 GB



A100: 80 GB



4 A100: 320 GB



Supera una tarjeta

La memoria decide el hardware

GB = gigabyte · GPU = unidad de procesamiento gráfico · FP16 = coma flotante de 16 bits · VRAM = memoria de la GPU

Fuente: cálculo propio (2 bytes por parámetro en FP16) con especificaciones de A10G y A100 de NVIDIA; caso ilustrativo

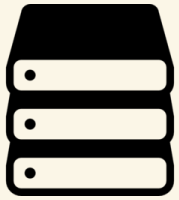
SÍNTESIS CONCEPTUAL

Cuatro principios de cómputo acelerado

Lo que se lleva de la primera parte



1 · Ancho de banda
Más GB/s



2 · Precisión
Menos bytes



3 · Interconexión
NVLink



4 · Particionamiento
Varias cargas

El hardware fija el techo

GB = gigabyte

Fuente: síntesis de las diapositivas 2 a 14 de esta parte

SEMANA 15 — INTELIGENCIA ARTIFICIAL EN LA NUBE II

IA en la Nube II: GPU, Cuantización y Despliegue

Parte 2 de 5 — Cuantización y Optimización de Modelos

Ing. Rodolfo Cañas Cervantes
Universidad de la Costa (CUC) — Ingeniería de Sistemas
Semana 15 — Periodo 2026-2 — Barranquilla, Colombia

Parte 2 de 5 — Cuantización y Optimización de Modelos
IA = inteligencia artificial · TCO = costo total de propiedad

Presupuesto de VRAM en inferencia

Pesos, caché KV y margen dentro de 24 GB

Memoria de una GPU de 24 GB



Pesos
16 GB

Caché KV
4 GB

Margen
1,6 GB

Reserva
2,4 GB

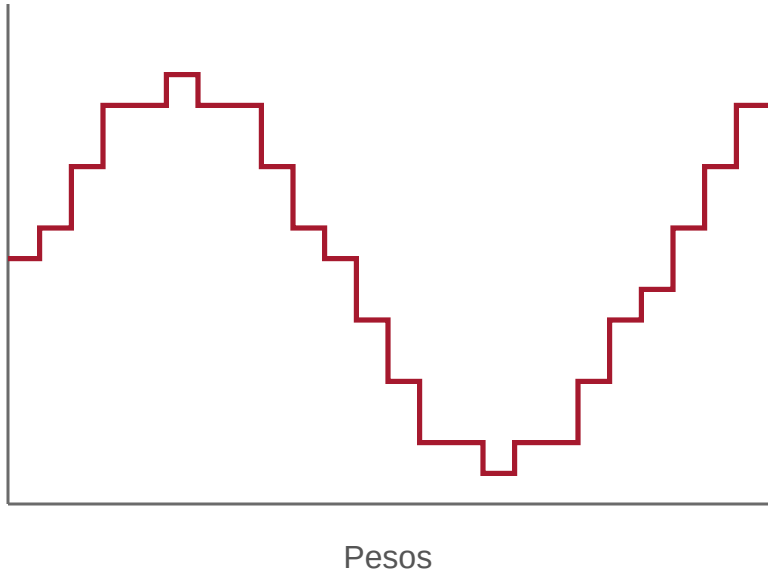
Suma ≤ 90 % de la VRAM

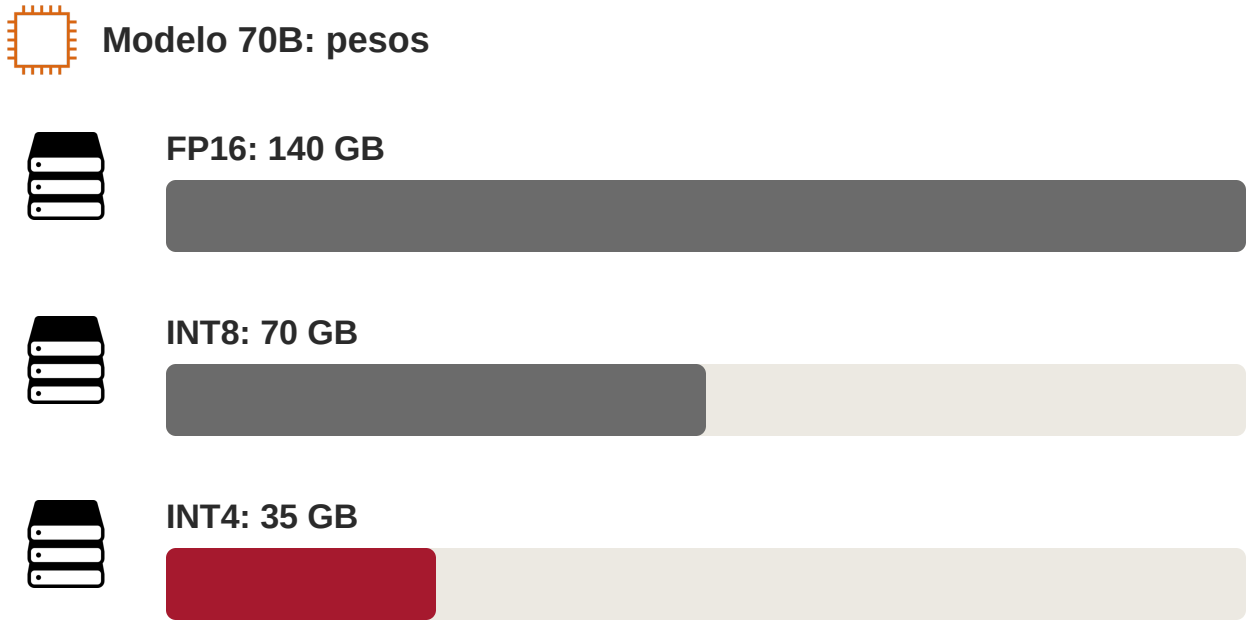
GPU = unidad de procesamiento gráfico · GB = gigabyte · KV = clave-valor (caché de atención) · VRAM = memoria de la GPU

Fuente: fórmula de memoria de pesos (cálculo propio); regla del 90 % de criterio de dimensionamiento del curso

Fundamentos de cuantización: de FP16 a enteros

Valores continuos a niveles discretos





A 4 bits: una cuarta parte

INT4 = entero de 4 bits · FP16 = coma flotante de 16 bits · GB = gigabyte · INT8 = entero de 8 bits

Fuente: cálculo propio (2 bytes frente a 0,5 bytes por parámetro) con el número de niveles de cada formato

PTQ frente a QAT

Cuantizar después o durante el entrenamiento

PTQ

Horas en una GPU



QAT

Días de cómputo



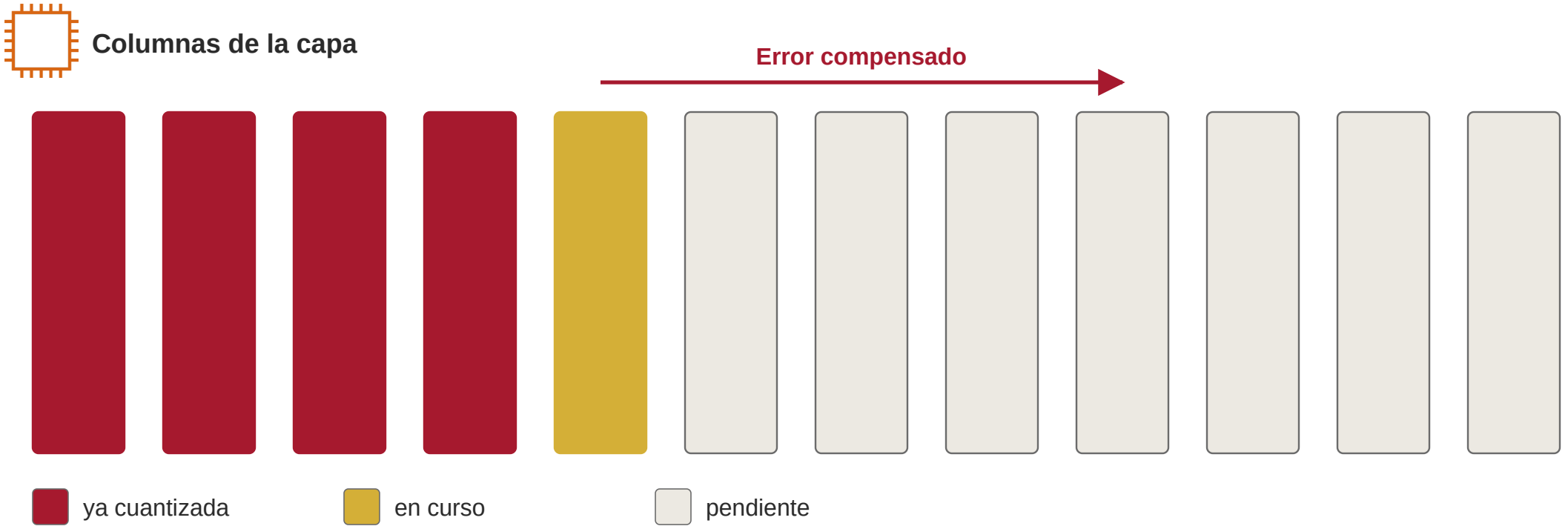
En producción domina PTQ (AWQ, GPTQ)

PTQ = cuantización posterior al entrenamiento · GPU = unidad de procesamiento gráfico · QAT = entrenamiento consciente de la cuantización · FP16 = coma flotante de 16 bits · INT4 = entero de 4 bits · AWQ = cuantización consciente de activaciones · GPTQ = cuantización por error residual por capas

Fuente: Jacob et al. (2018), arXiv:1712.05877; Gholami et al. (2021), arXiv:2103.13630

Algoritmo GPTQ: compensación de error

Cada columna corrige a las que siguen



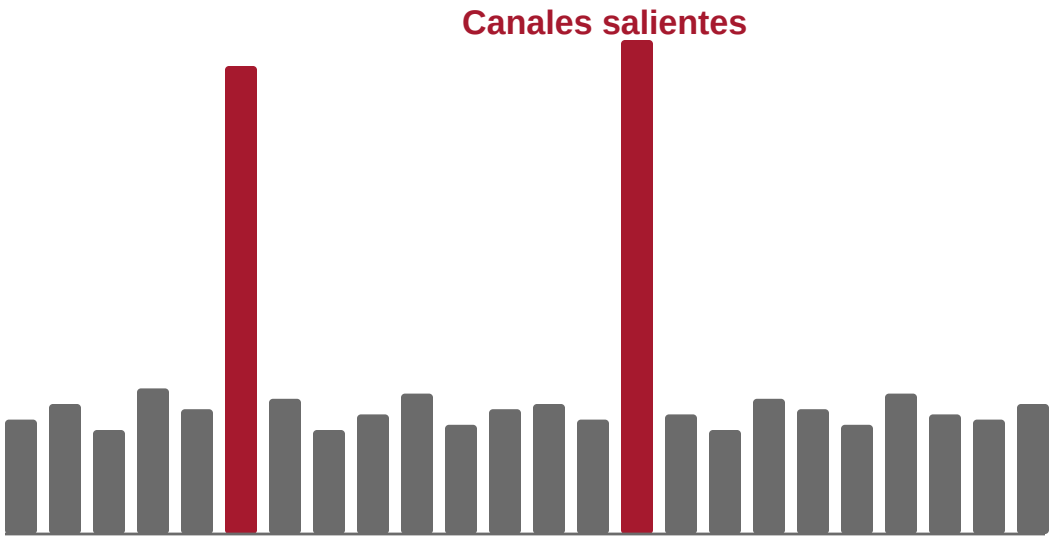
El error se compensa en las columnas pendientes

GPTQ = cuantización por error residual por capas

Fuente: Frantar et al. (2022), GPTQ, arXiv:2210.17323

Algoritmo AWQ: canales salientes

Medir las activaciones reales antes de cuantizar



Activación por canal

Uniforme
Error amplificado en todos los canales

AWQ
Escala los canales salientes

Proteger ~1 % de los canales

AWQ = cuantización consciente de activaciones

Fuente: Lin et al. (2023), AWQ, arXiv:2306.00978

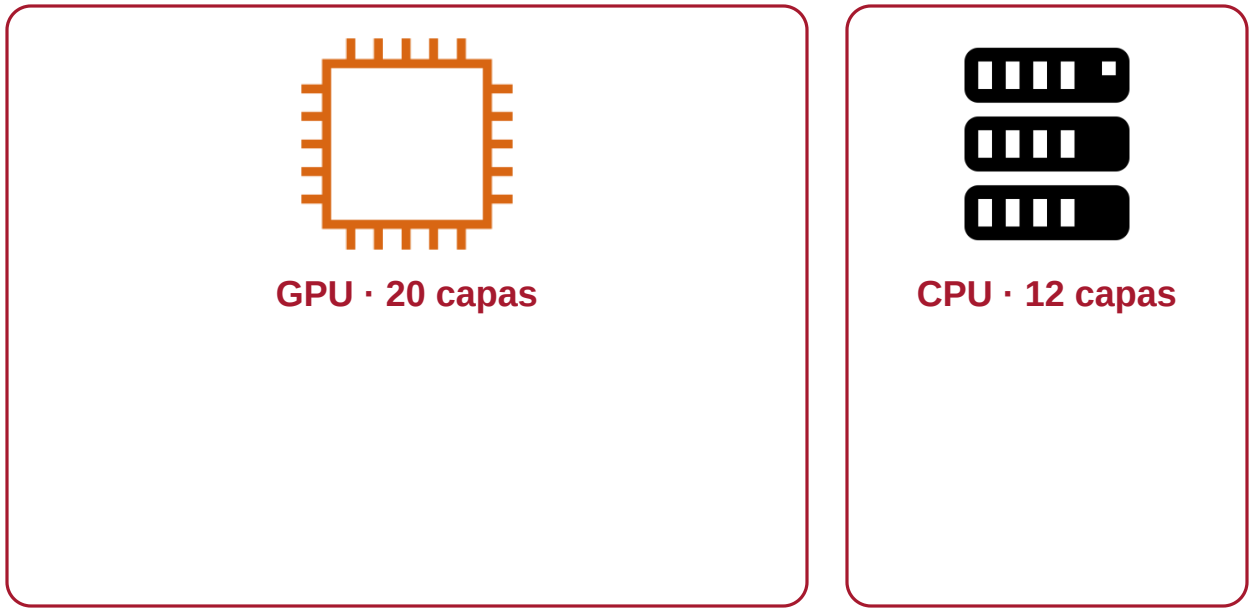
Formato GGUF y ecosistema llama.cpp

Un archivo autocontenido para llama.cpp



Archivo .gguf

Modelo de 32 capas



Portable en CPU y GPU

GPU = unidad de procesamiento gráfico · CPU = unidad central de procesamiento · GGUF = formato binario autocontenido de llama.cpp

Fuente: documentación de GGUF y llama.cpp (github.com/ggerganov/llama.cpp)

Métricas de degradación: perplejidad y benchmarks

Pruebas de aceptación antes de desplegar



Perplejidad (PPL)
Sube poco



MMLU
Cerca del FP16



GSM8K
Sensible a errores

Mismas pruebas sobre el modelo FP16 y el cuantizado

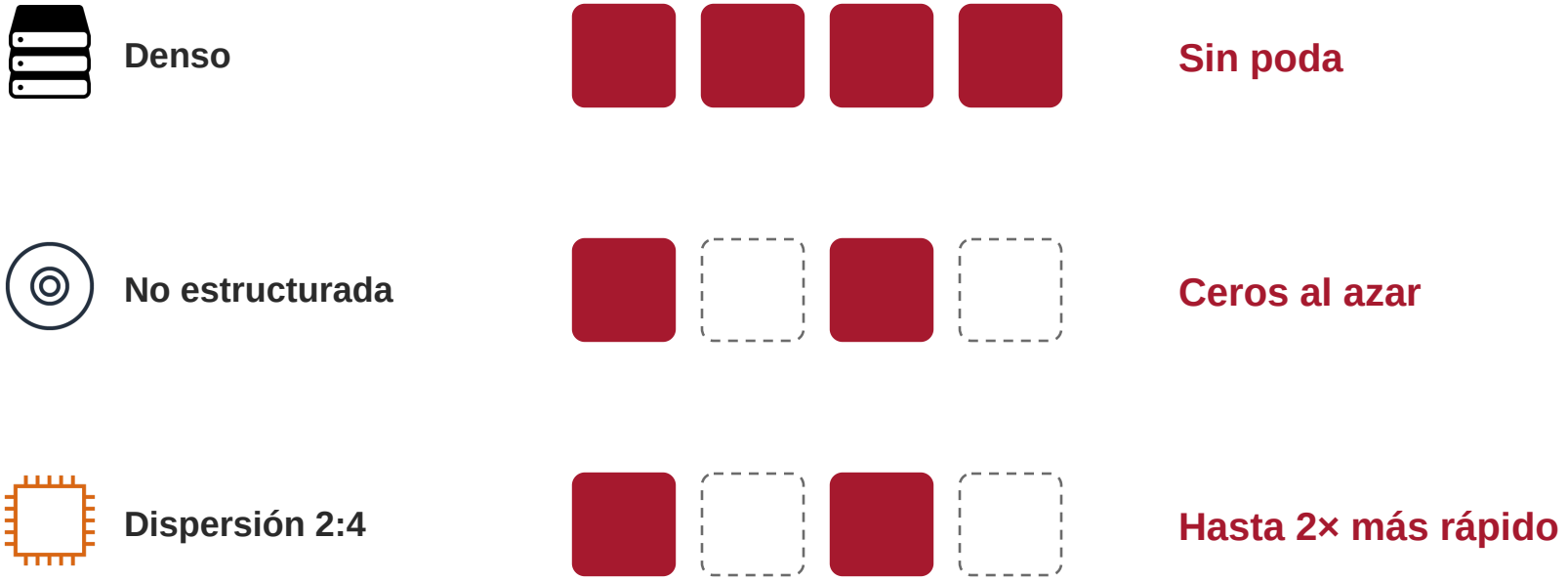
Mirar solo la perplejidad no basta

MMLU = examen masivo multitarea · FP16 = coma flotante de 16 bits

Fuente: Hendrycks et al. (2021), MMLU, arXiv:2009.03300; Cobbe et al. (2021), GSM8K, arXiv:2110.14168

Poda y dispersión estructurada 2:4

Podar sirve si el hardware salta los ceros

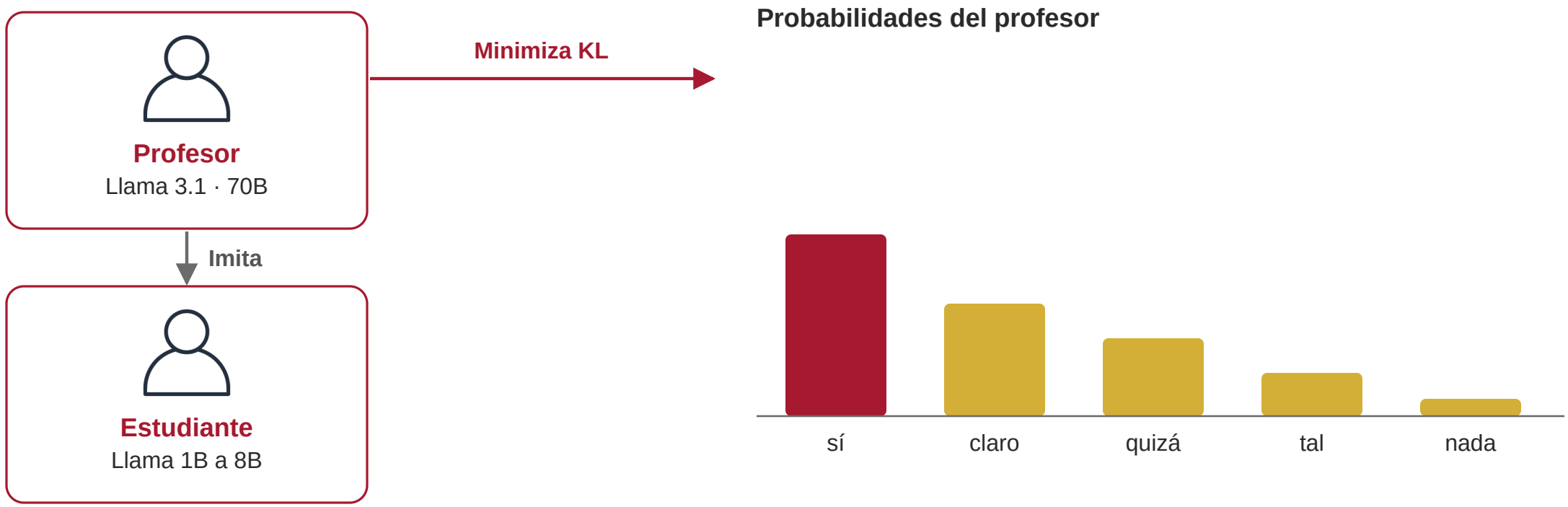


Solo acelera con hardware compatible

Fuente: NVIDIA, Structured Sparsity in Ampere and Hopper (developer.nvidia.com/blog)

Destilación de conocimiento: profesor a estudiante

El estudiante imita la distribución completa



Aprende qué opciones son plausibles

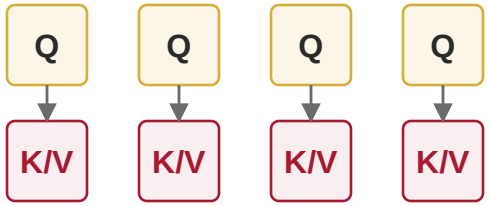
KL = divergencia de Kullback-Leibler

Fuente: Hinton et al. (2015), arXiv:1503.02531; Meta, Llama 3.1 model card (llama.meta.com)

Evolución de la atención: MHA, MQA y GQA

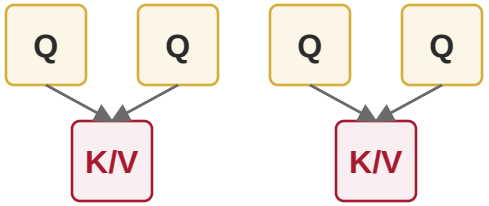
Compartir pares K/V reduce la caché

MHA



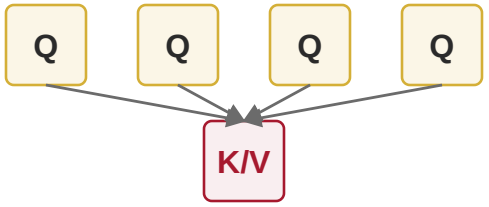
Caché: 4 pares

GQA



Caché: 2 pares

MQA



Caché: 1 par

GQA: equilibrio común

MHA = atención multicabeza · GQA = atención de consultas agrupadas · MQA = atención de consulta múltiple

Fuente: Shazeer (2019), MQA, arXiv:1911.02150; Ainslie et al. (2023), GQA, arXiv:2305.13245

Cuantización de la KV-Cache: FP8 e INT8

El KV en FP8 duplica las sesiones



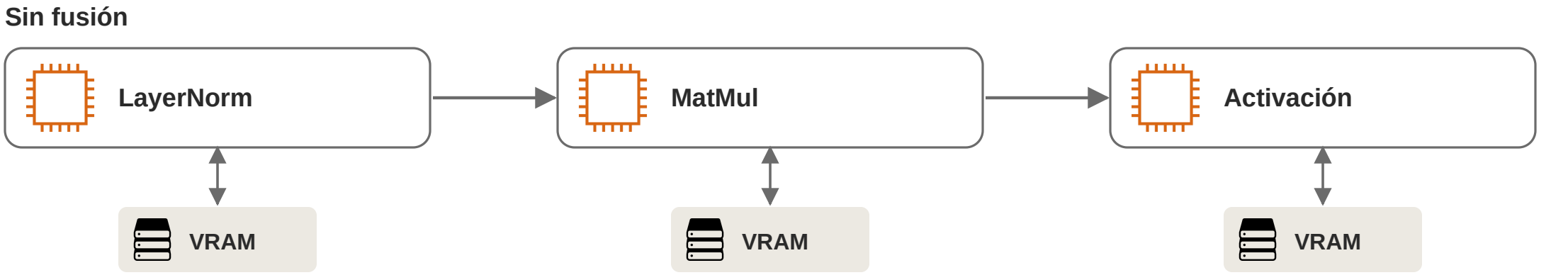
Más sesiones, mismos pesos

FP16 = coma flotante de 16 bits · GB = gigabyte · FP8 = coma flotante de 8 bits · KV = clave-valor (caché de atención) · INT8 = entero de 8 bits

Fuente: cálculo propio con configuración de Llama 3 8B (Dubey et al. 2024, arXiv:2407.21783); Kwon et al. (2023), arXiv:2309.06180

Compiladores de inferencia: TensorRT-LLM

Fusionar kernels evita escribir en VRAM



Menos viajes a la VRAM

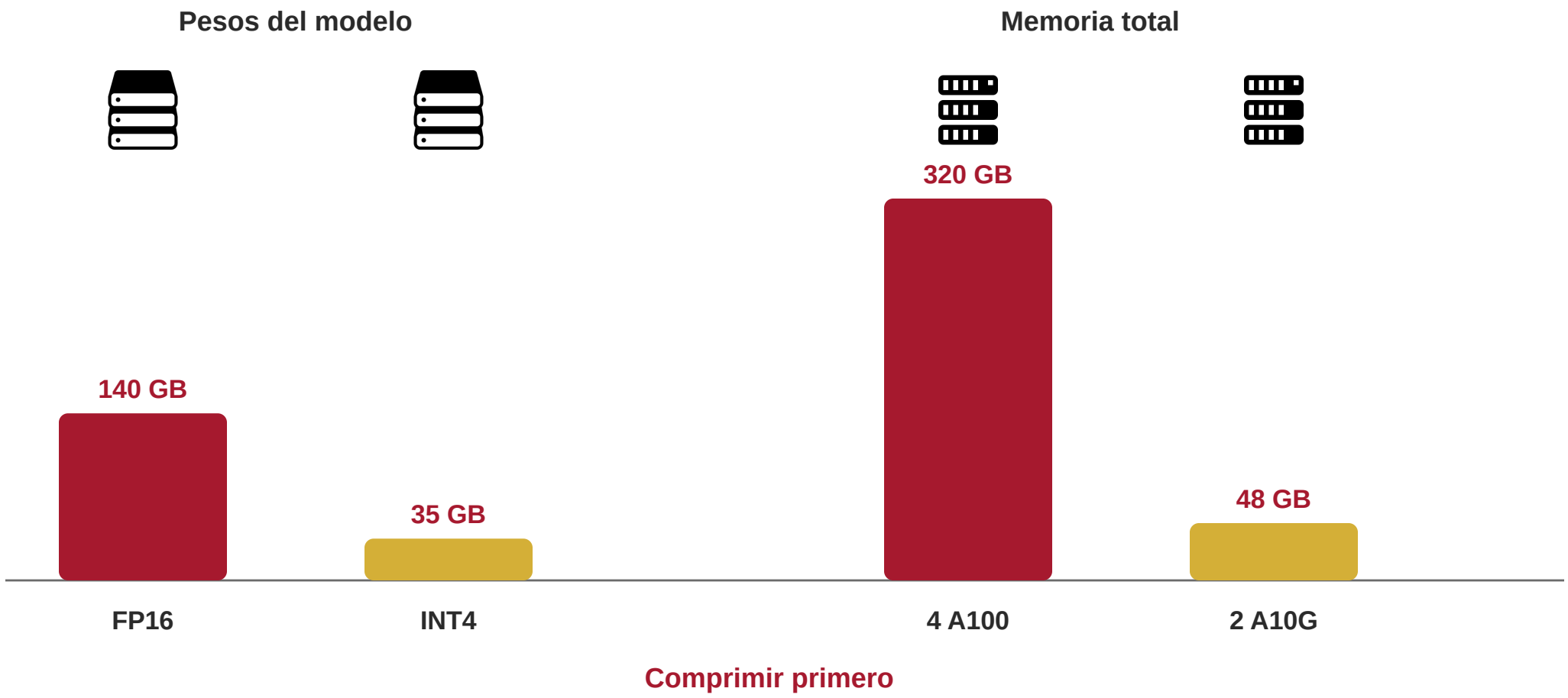
VRAM = memoria de la GPU · LLM = modelo de lenguaje de gran tamaño

Fuente: NVIDIA, TensorRT-LLM (github.com/NVIDIA/TensorRT-LLM); Dao et al. (2022), FlashAttention, arXiv:2205.14135

CASO ILUSTRATIVO

Compresión de Llama de 70B

Comprimir cambia el hardware necesario



GB = gigabyte · FP16 = coma flotante de 16 bits · INT4 = entero de 4 bits

Fuente: cálculo propio (bytes por parámetro); especificaciones de A100 y A10G (NVIDIA); caso ilustrativo

Comparativa de métodos de cuantización

Cada método se elige por el entorno

Método	Fuerte	Entorno
AWQ (INT4)	Canales salientes	vLLM, GPU de nube
GPTQ (INT4)	Compensación de error	AutoGPTQ, vLLM
GGUF	Archivo único	llama.cpp, Ollama
FP8	Coma flotante de 8 bits	TensorRT-LLM, Hopper

Elegir el método según el entorno de despliegue

AWQ = cuantización consciente de activaciones · vLLM = motor de inferencia de código abierto · GPU = unidad de procesamiento gráfico · GPTQ = cuantización por error residual por capas · GGUF = formato binario autocontenido de llama.cpp · FP8 = coma flotante de 8 bits · LLM = modelo de lenguaje de gran tamaño

Fuente: síntesis de las diapositivas 19 a 29 de esta parte; motores según su documentación oficial

SEMANA 15 — INTELIGENCIA ARTIFICIAL EN LA NUBE II

IA en la Nube II: GPU, Cuantización y Despliegue

Parte 3 de 5 — Servidores de Inferencia Auto-Alojados (Self-Hosted)

Ing. Rodolfo Cañas Cervantes
Universidad de la Costa (CUC) — Ingeniería de Sistemas
Semana 15 — Periodo 2026-2 — Barranquilla, Colombia

Parte 3 de 5 — Servidores de Inferencia Auto-Alojados (Self-Hosted)
IA = inteligencia artificial · TCO = costo total de propiedad

Anatomía de un servidor de inferencia

Cuatro capas entre la petición y la GPU



Cada capa desacopla la petición del silicio

API = interfaz de programación · vLLM = motor de inferencia de código abierto · GPU = unidad de procesamiento gráfico · KV = clave-valor (caché de atención)

Fuente: documentación de vLLM (docs.vllm.ai)

vLLM y PagedAttention: sin fragmentación

vLLM asigna bloques de 16 tokens

Reserva contigua

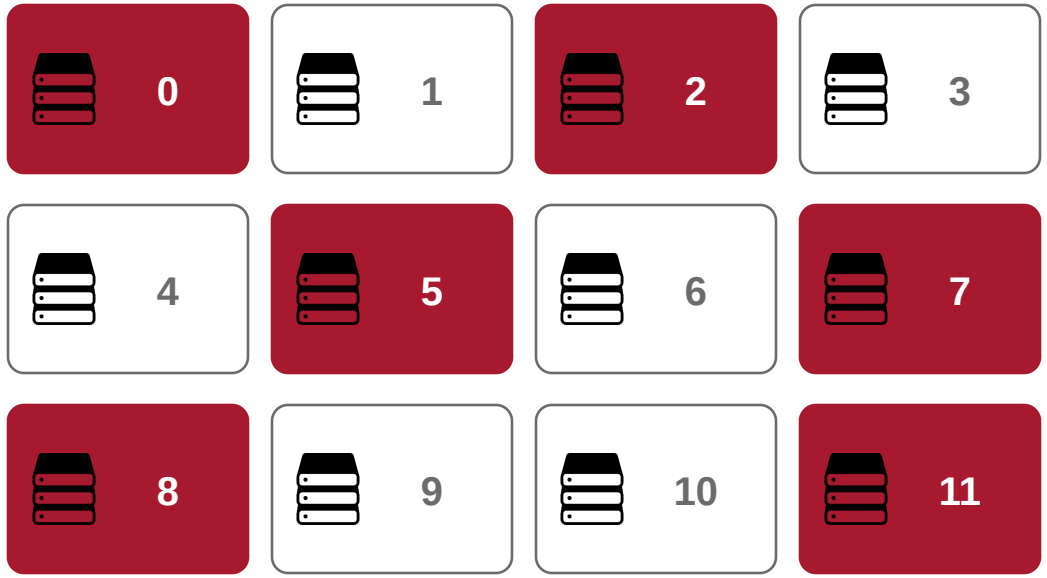
Bloquea 4 096 tokens

Usados: 200

Bloques de 16
Se asignan al crecer

Sin huecos grandes
Se reutilizan por bloque

Bloques físicos



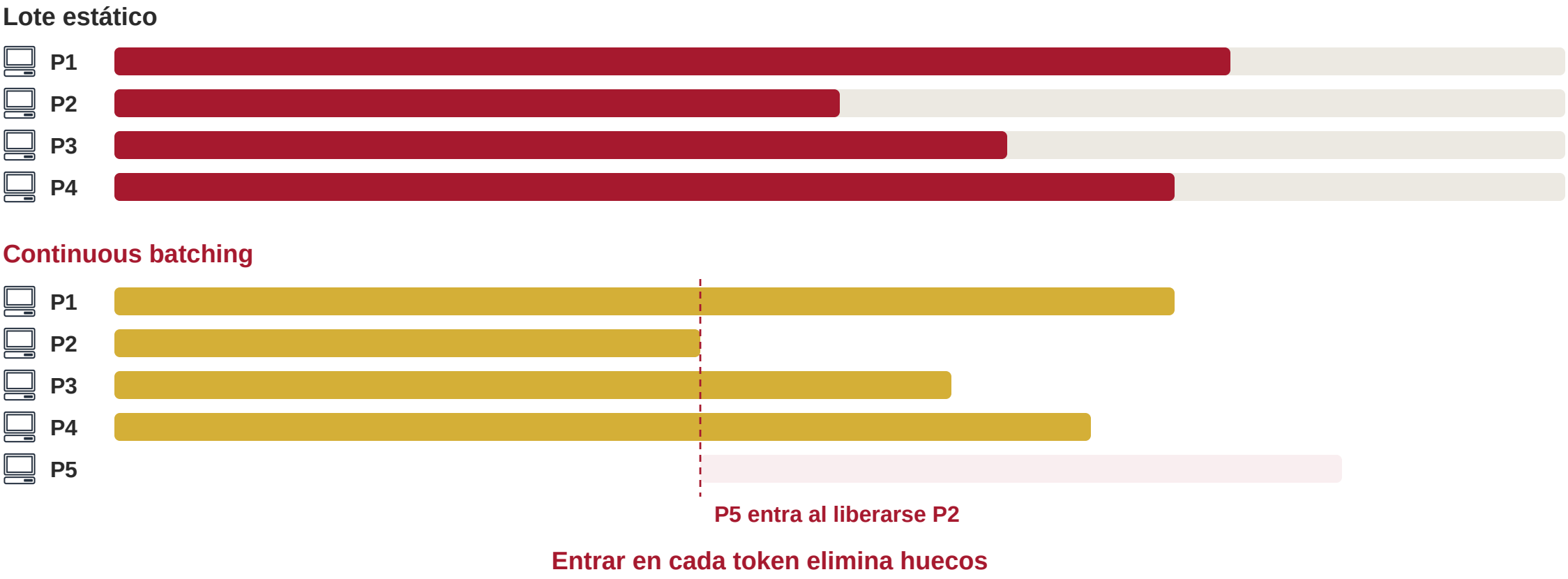
Como la memoria virtual

vLLM = motor de inferencia de código abierto

Fuente: Kwon et al. (2023), PagedAttention, arXiv:2309.06180

Continuous batching (in-flight batching)

Planificar por token, no por lote

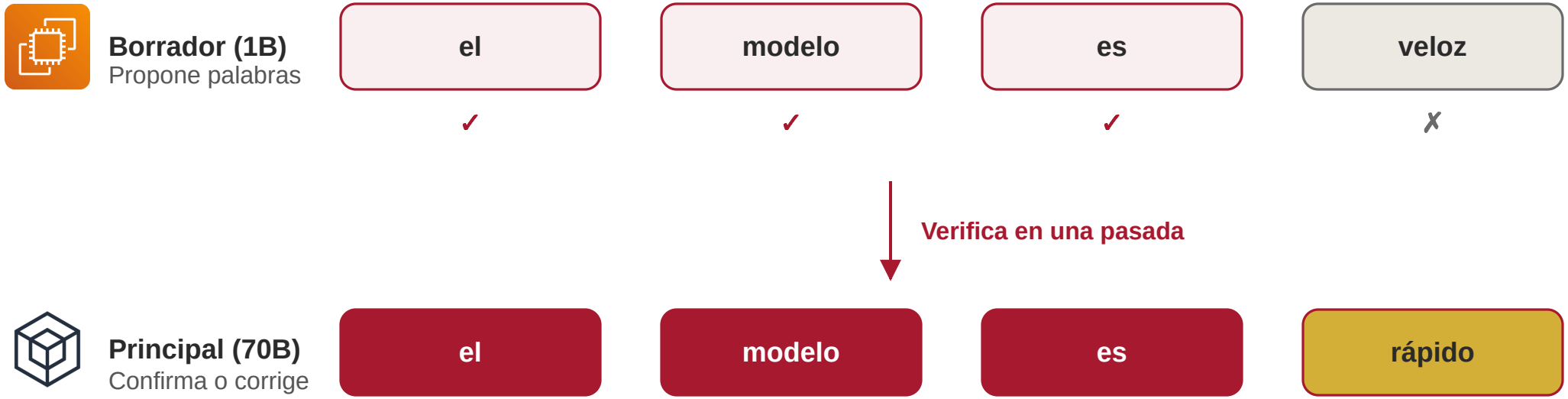


Entrar en cada token elimina huecos

Fuente: Yu et al. (2022), Orca (OSDI); Kwon et al. (2023), arXiv:2309.06180

Decodificación especulativa

Un modelo pequeño propone; uno grande verifica

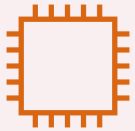


Misma distribución, 2× a 3× más rápido

Fuente: Leviathan et al. (2023), arXiv:2211.17192

Comparativa de motores: vLLM, TGI y Triton

Comparativa de motores de inferencia



vLLM
Muchos usuarios



TGI
Paradas estrictas



Triton
Pipelines NVIDIA



TensorRT-LLM
Modelos heterogéneos

Cuándo elegirlo: el patrón de tráfico

El patrón de tráfico elige el motor

vLLM = motor de inferencia de código abierto · TGI = servidor de inferencia de Hugging Face · NVIDIA = fabricante de GPU · LLM = modelo de lenguaje de gran tamaño

Fuente: documentación de vLLM, TGI (huggingface.co/docs/text-generation-inference) y NVIDIA Triton

Despliegue en Kubernetes con GPU Operator y KEDA

Cuatro piezas para escalar inferencia



GPU Operator
Expone nvidia.com/gpu



Pods de vLLM
Probes y pesos montados



KEDA
Escala por cola



Karpenter
Nodos GPU en minutos

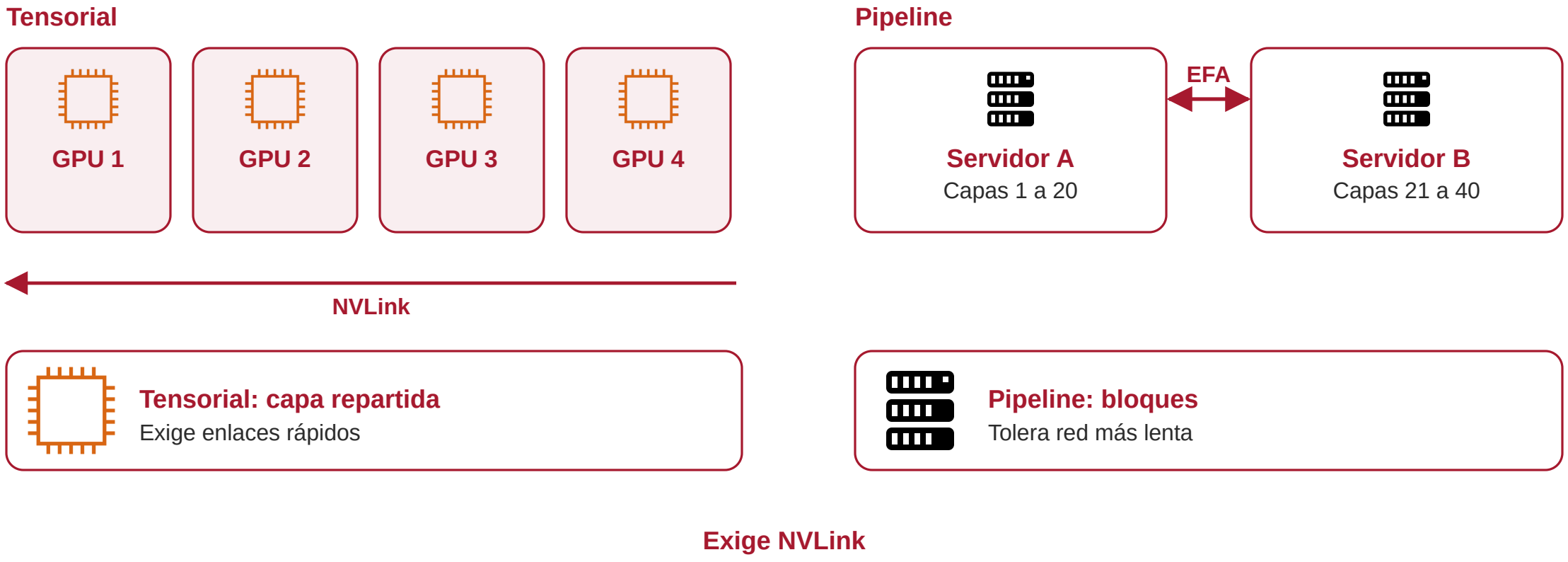
Escalar por cola, no por CPU

GPU = unidad de procesamiento gráfico · vLLM = motor de inferencia de código abierto · KEDA = autoescalado dirigido por eventos · CPU = unidad central de procesamiento

Fuente: NVIDIA GPU Operator; KEDA (keda.sh); Karpenter (karpenter.sh)

Paralelismo tensorial y de pipeline

Partir la capa o partir las capas



GPU = unidad de procesamiento gráfico · EFA = adaptador de red elástico de Amazon Web Services

Fuente: Shoeybi et al. (2019), Megatron-LM, arXiv:1909.08053; Meta, Llama 3.1 model card

Métricas de servicio: TTFT, TPOT y throughput

Indicadores que definen el SLA



Un token por paso

TTFT
Primer token

TPOT
Cadencia de lectura

Throughput
Tokens por segundo

TTFT < 800 ms (supuesto)

TTFT = tiempo hasta el primer token · TPOT = tiempo por token de salida · SLA = acuerdo de nivel de servicio

Fuente: definiciones de TTFT y TPOT (documentación de monitoreo de inferencia); objetivos supuestos

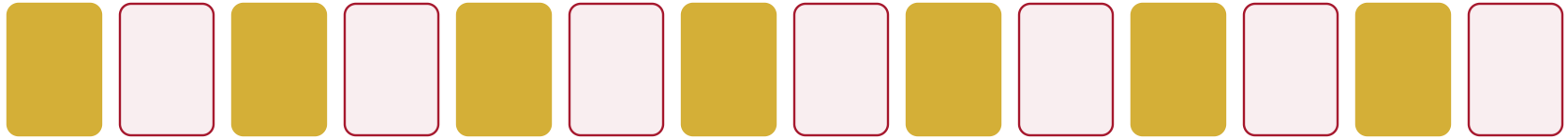
Control de latencia P99: chunked prefill

Trocear el prompt reparte la GPU

Prefill monolítico

Prompt de 16 000 tokens

Trozos de 512



Trozo de prefill

Token de decodificación

Latencia P99 estable, sin pausas

GPU = unidad de procesamiento gráfico

Fuente: Agrawal et al. (2023), SARATHI, arXiv:2308.16369

vLLM en nube privada OpenStack

Cómputo, almacenamiento y red en OpenStack



Centro de cómputo

Nova
Cómputo con GPU

Cinder
Volúmenes NVMe

Neutron
Red privada aislada

Octavia
Balanceo local

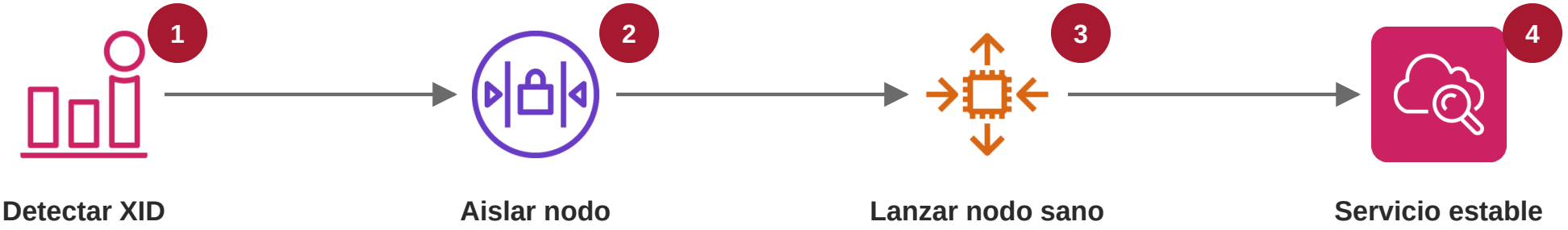
Los prompts no salen del centro de datos

GPU = unidad de procesamiento gráfico · vLLM = motor de inferencia de código abierto

Fuente: documentación de OpenStack (Nova, Cinder, Neutron, Octavia) · Foto: Rubin Observatory/NSF/AURA, Wikimedia Commons (CC BY 4.0)

Recuperación de fallos y errores XID de GPU

Errores de GPU: detectar, aislar, reemplazar



XID 31
Fallo de memoria de la GPU

XID 79
La GPU se cayó del bus

Reemplazar el nodo

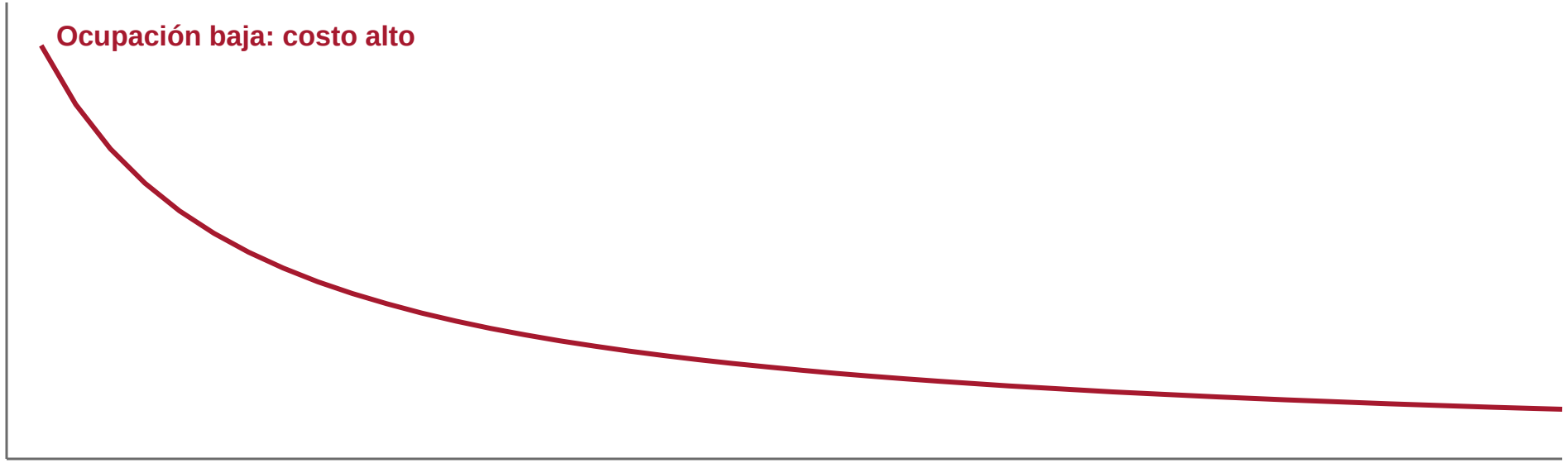
XID = código de error de GPU de Nvidia · GPU = unidad de procesamiento gráfico

Fuente: NVIDIA, catálogo de errores XID (docs.nvidia.com/deploy/xid-errors)

Auditoría de tokens y FinOps en IA

Atribución de consumo por API key y gateway

Costo por token



El costo por token depende de la ocupación

IA = inteligencia artificial · API = interfaz de programación

Fuente: FinOps Foundation, FinOps for AI (finops.org); curva de costo, cálculo propio

CASO ILUSTRATIVO

Arquitectura self-hosted de producción

Diseño de referencia con 4 nodos A10G



Racks

Ingress y TLS

↓

Enrutador

↓

4 nodos vLLM
A10G, AWQ

↓

Telemetría

TTFT < 350 ms (supuesto)

TLS = seguridad de capa de transporte · vLLM = motor de inferencia de código abierto · AWQ = cuantización consciente de activaciones · TTFT = tiempo hasta el primer token

Fuente: diseño de referencia ilustrativo; Llama 3 (Meta) · Foto: PiDatacenters, Wikimedia Commons (CC BY-SA 4.0)

Cuatro reglas de operación para inferencia auto-alojada

Cuatro reglas para inferencia auto-alojada

1 · Memoria VRAM
Utilización de 0,90

2 · Planificación
Batching y chunked prefill

3 · Telemetría
DCGM y KEDA

4 · Resiliencia
Drenado automático

Configurar antes de recibir tráfico

VRAM = memoria de la GPU · DCGM = monitor de GPU de Nvidia · KEDA = autoescalado dirigido por eventos

Fuente: síntesis de las diapositivas 32 a 44 de esta parte

SEMANA 15 — INTELIGENCIA ARTIFICIAL EN LA NUBE II

IA en la Nube II: GPU, Cuantización y Despliegue

Parte 4 de 5 — Servicios Gestionados vs Auto-Alojados y TCO

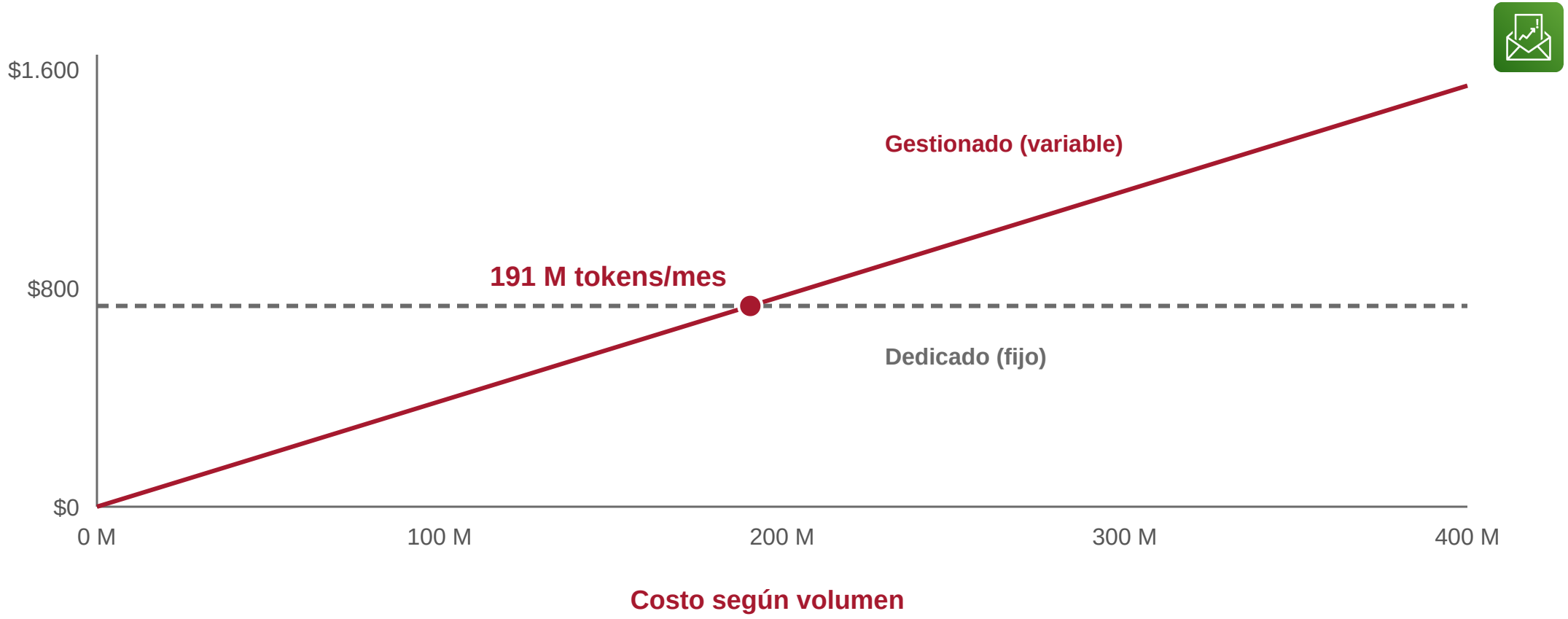
Ing. Rodolfo Cañas Cervantes
Universidad de la Costa (CUC) — Ingeniería de Sistemas
Semana 15 — Periodo 2026-2 — Barranquilla, Colombia

Parte 4 de 5 — Servicios Gestionados vs Auto-Alojados y TCO
IA = inteligencia artificial · TCO = costo total de propiedad

DILEMA ARQUITECTÓNICO

Gestionado frente a auto-alojado

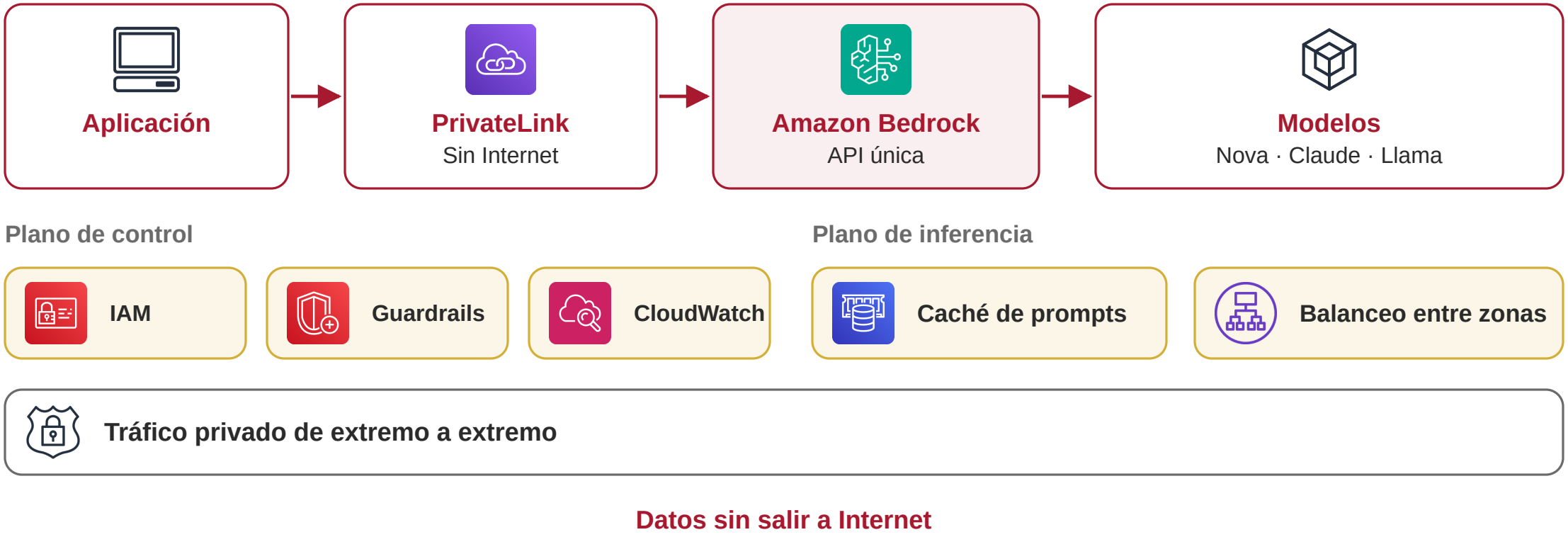
Costo mensual según el volumen de tokens



Fuente: cálculo propio con el punto de equilibrio de la diapositiva 54; tarifas de Amazon Bedrock (archivo local, consulta 2026-11-05)

Arquitectura de Amazon Bedrock

Red privada con inferencia gestionada



API = interfaz de programación · IAM = gestión de identidades y accesos

Fuente: AWS, Amazon Bedrock User Guide y AWS PrivateLink (docs.aws.amazon.com/bedrock)

On-demand frente a capacidad aprovisionada

Pagar por token o reservar capacidad

On-demand
Pago por token, sin compromiso

Aprovisionado
Tarifa fija por hora reservada

Tráfico irregular → **On-demand**

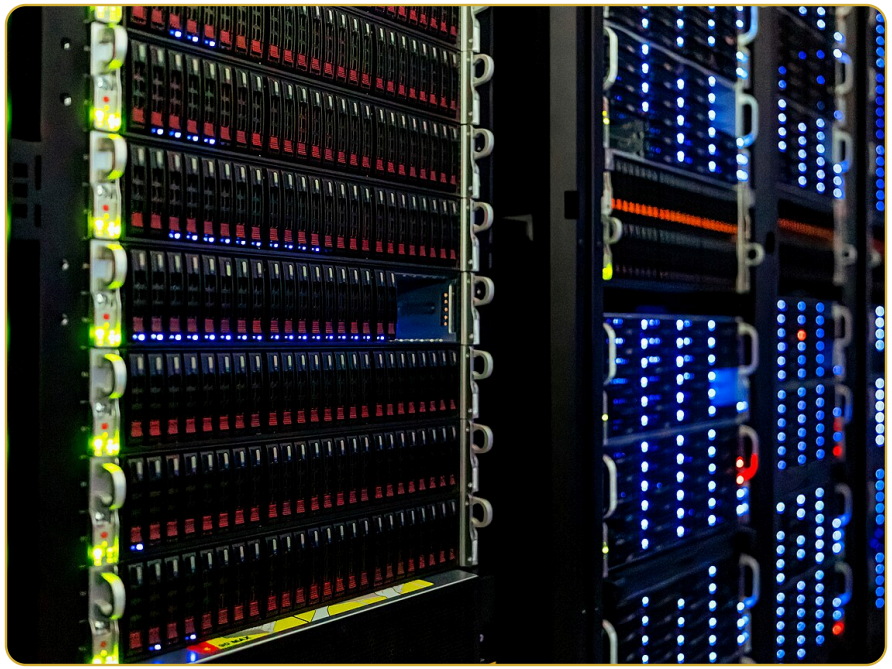
Tráfico sostenido → **Aprovisionar**

Latencia crítica: aprovisionar

Fuente: AWS, Bedrock quotas y provisioned throughput (docs.aws.amazon.com/bedrock)

Aislamiento y retención cero de datos (ZDR)

Garantías técnicas en cada tramo



Sala de servidores

TLS 1.3 en tránsito

↓

PrivateLink sin Internet

↓

Cifrado con KMS

↓

ZDR: sin retención

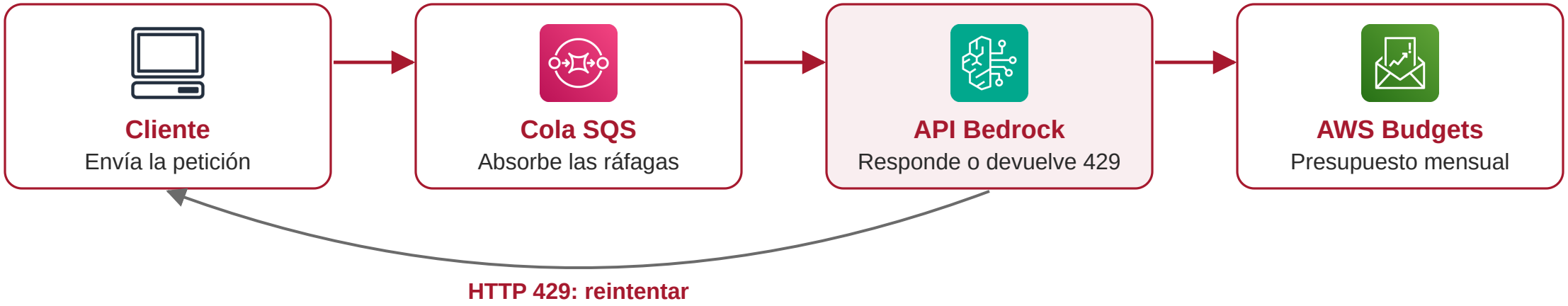
Confianza con controles verificables

TLS = seguridad de capa de transporte · KMS = servicio de gestión de claves

Fuente: AWS, Bedrock data protection (docs.aws.amazon.com/bedrock) · Foto: NOIRLab/NSF/AURA, Wikimedia Commons (CC BY 4.0)

Cuotas, límites de tasa y reintentos

Cuotas, reintentos y presupuesto



Topes RPM y TPM
Límites por cuenta

Alerta al 80 %
Aviso antes del tope

El 429 es diseño

SQS = servicio de colas simples · API = interfaz de programación · AWS = Amazon Web Services · HTTP = protocolo web · RPM = peticiones por minuto · TPM = tokens por minuto

Fuente: AWS, Bedrock quotas; Amazon SQS; AWS Budgets (documentación oficial)

Responsabilidad compartida en inteligencia artificial

Quién responde por cada capa



Cliente — Datos, IAM y salidas



Creador del modelo — Curaduría y alineación



AWS — Infraestructura y SLA

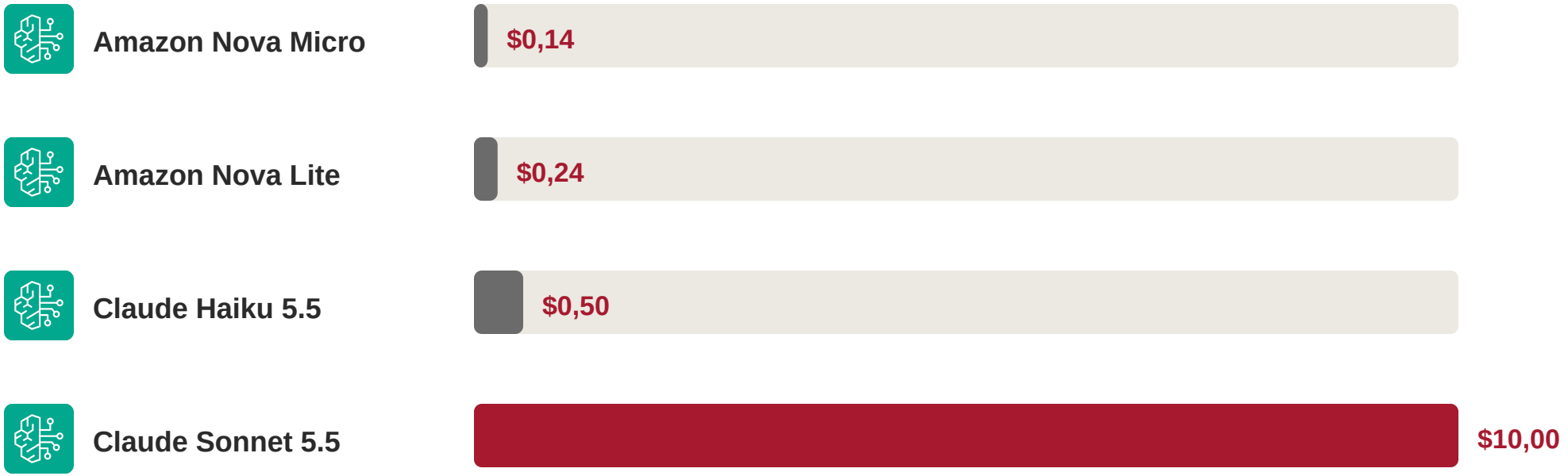
Contratar un modelo no traslada la responsabilidad

IAM = gestión de identidades y accesos · AWS = Amazon Web Services · SLA = acuerdo de nivel de servicio

Fuente: AWS, Shared Responsibility Model for generative AI (aws.amazon.com/compliance/shared-responsibility-model)

Estructura de precios en Amazon Bedrock

Precio por millón de tokens de salida



La salida de Sonnet cuesta 20 veces la de Haiku

Fuente: tarifas de Amazon Bedrock us-east-1 (archivo local, consulta 2026-11-05)

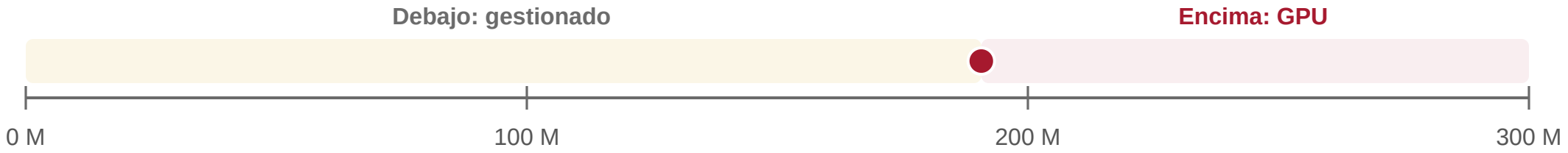
Punto de equilibrio: gestionado frente a GPU

Cuántos tokens justifican un servidor dedicado

$734 / mes
Servidor g5.xlarge

$3,85 / M
Costo medio

191 M tokens
Equilibrio



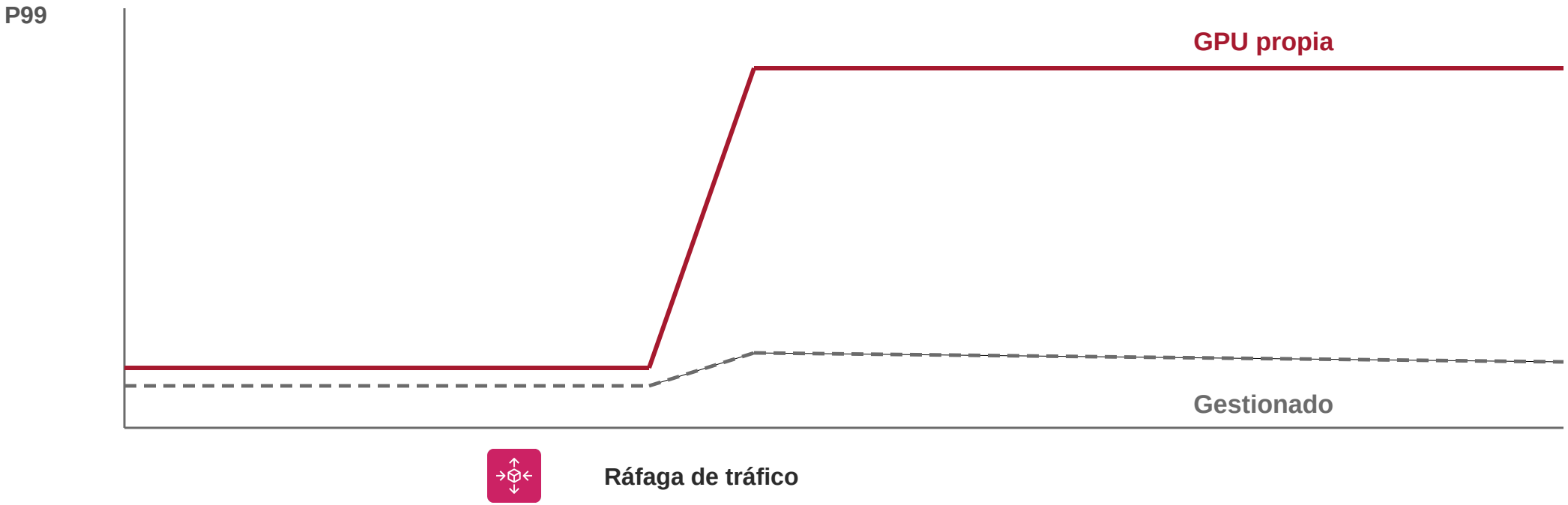
Por encima de 191 M, GPU propia

GPU = unidad de procesamiento gráfico · TCO = costo total de propiedad

Fuente: cálculo propio: $734,38 (= $1,006 × 730 h) ÷ $3,85 por M (Sonnet 5.5; petición de 1 300 tokens: 1 000 entrada + 300 salida = $0,005)

Perfil de latencia y SLA: gestionado frente a GPU

Perfil de latencia ante una ráfaga



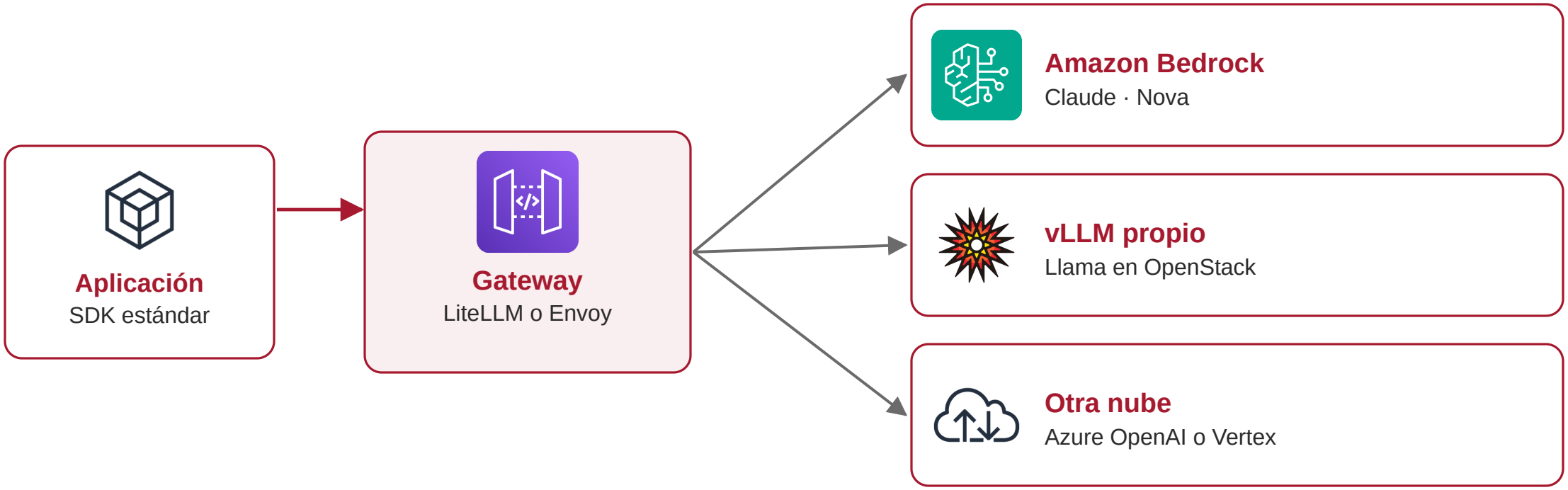
La GPU propia necesita margen de capacidad

GPU = unidad de procesamiento gráfico · SLA = acuerdo de nivel de servicio

Fuente: esquema pedagógico; SLA de Amazon Bedrock (documentación oficial)

Mitigación del bloqueo de proveedor

Una pasarela independiente del backend



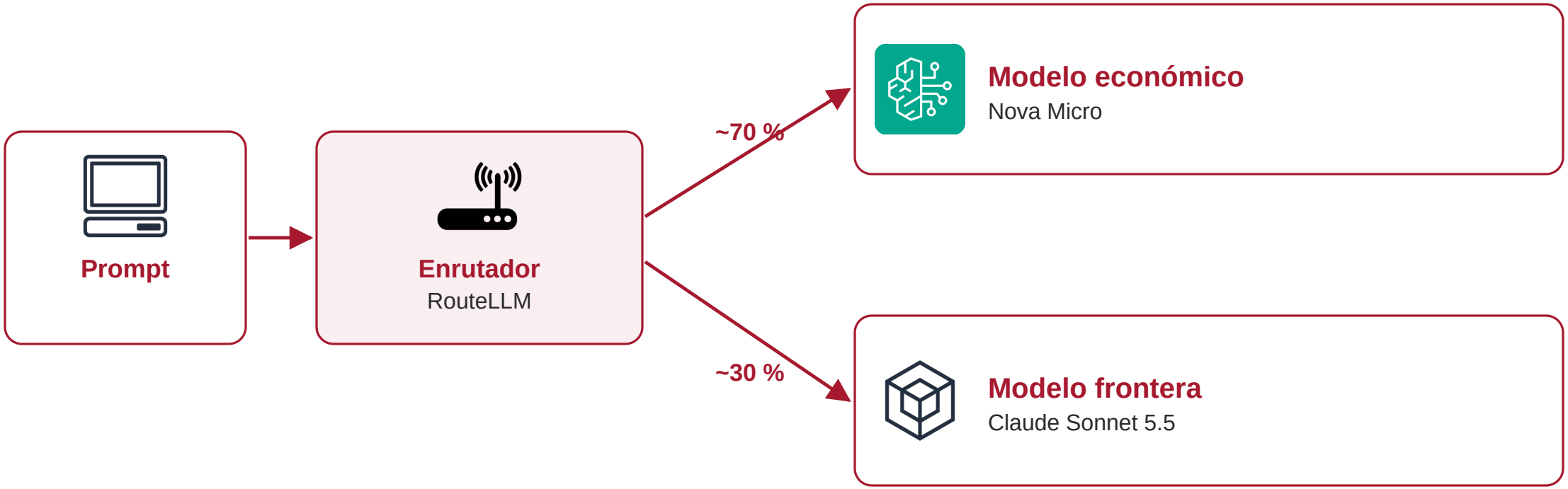
Cambiar de proveedor: una regla, no código

SDK = kit de desarrollo de software · vLLM = motor de inferencia de código abierto

Fuente: LiteLLM (docs.litellm.ai); Envoy Proxy (envoyproxy.io)

Ruteo dinámico inteligente de inferencia

Ruteo dinámico: porcentajes supuestos

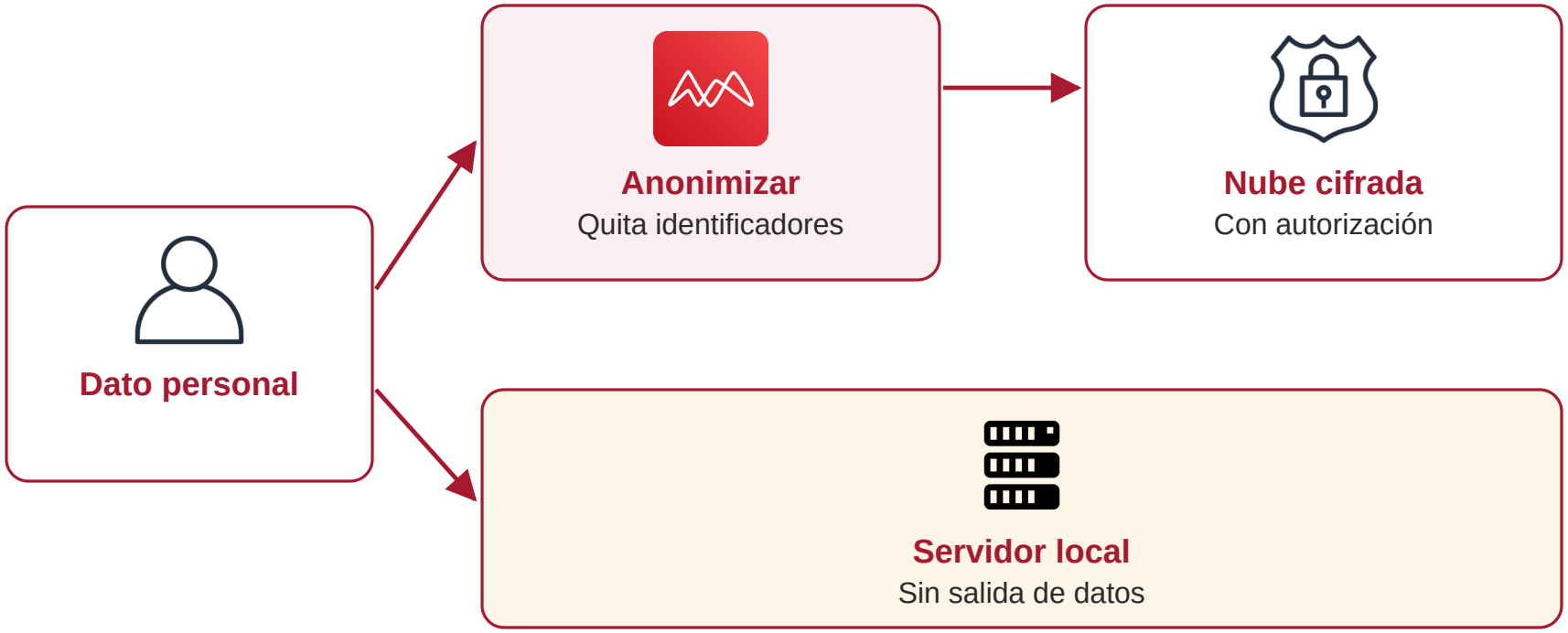


Clasificar antes de elegir el modelo

Fuente: Ong et al. (2024), RouteLLM, arXiv:2406.18665

Soberanía de datos y cumplimiento en Colombia

Qué dato puede ir a qué lugar



Saber qué dato va a dónde

Fuente: Ley 1581 de 2012, art. 26 (Gestor Normativo, Función Pública); SFC, Circular Externa 007 de 2018

Matriz de decisión: gestionado o auto-alojado

Tres modalidades según el perfil del proyecto

	Gestionado	Híbrido	Auto-alojado
Perfil	Pilotos y RAG	Tráfico heterogéneo	Datos sensibles
Ventaja	Cero mantenimiento	Costo optimizado	Control total
Umbral	< 50 M tokens/mes	Según el ruteo	Tráfico masivo

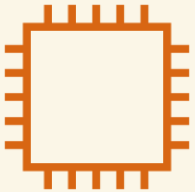
El perfil decide la modalidad

RAG = generación aumentada por recuperación

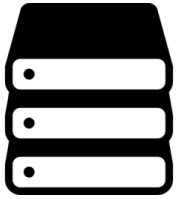
Fuente: criterios de la guía del curso; cálculo propio con tarifas de Bedrock

Conclusiones de la parte 4

Cuatro ideas de la parte 4



1. Silicio
HBM3 y NVLink



2. Cuantización
AWQ, GPTQ, GQA



3. Motores
PagedAttention y batching



4. Gestión
Gestionado o propio

Sigue la parte 5

HBM3 = memoria de alto ancho de banda, 3.ª generación · AWQ = cuantización consciente de activaciones · GPTQ = cuantización por error residual por capas · GQA = atención de consultas agrupadas · IA = inteligencia artificial · III = tercera (número romano)

SEMANA 15 — INTELIGENCIA ARTIFICIAL EN LA NUBE II

IA en la Nube II: GPU, Cuantización y Despliegue

Parte 5 de 5 — Elegir y Combinar Modelos: IA Local, MoE y Delegación

Ing. Rodolfo Cañas Cervantes
Universidad de la Costa (CUC) — Ingeniería de Sistemas
Semana 15 — Periodo 2026-2 — Barranquilla, Colombia

Parte 5 de 5 — Elegir y Combinar Modelos: IA Local, MoE y Delegación
IA = inteligencia artificial · MoE = mezcla de expertos

Correr modelos abiertos en tu propio equipo

Tres herramientas para un modelo cuantizado



Sin nube: el modelo se ejecuta en tu equipo

LM = modelo de lenguaje (language model) · GGUF = formato binario autocontenido de llama.cpp · IA = inteligencia artificial

Fuente: ollama.com, github.com/ggml-org/llama.cpp y lmstudio.ai (páginas oficiales, consulta 2026-11-05)

Cuándo conviene correr el modelo en local

Privacidad y práctica frente a escala y calidad máxima



GPU de consumo (RTX 4090)

Conviene

- Datos que no deben salir
- Trabajo sin conexión
- Práctica sin costo por token

No conviene

- Modelos muy grandes
- Muchos usuarios a la vez
- Máxima calidad

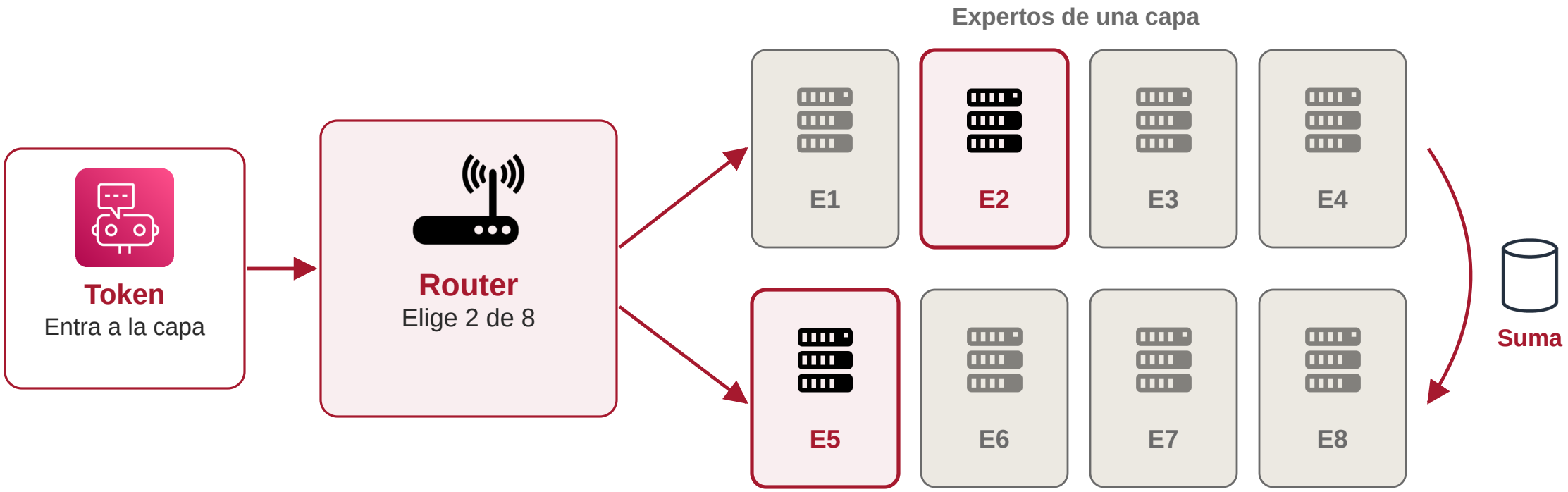
Local para privacidad y práctica; nube para escala

GPU = unidad de procesamiento gráfico · RTX = serie de GPU de consumo · IA = inteligencia artificial

Fuente: criterio docente del curso · Foto: Benlisquare, Wikimedia Commons (CC BY-SA 4.0)

MoE (mezcla de expertos): un router elige

Cada token activa solo una parte del modelo



Mixtral 8x7B: 8 expertos por capa, 2 por token

MoE = mezcla de expertos

Fuente: Jiang et al. (2024), Mixtral of Experts, arXiv:2401.04088 (consulta 2026-11-05)

Parámetros totales frente a activos

Miles de millones de parámetros (B) por token

Mixtral 8x7B

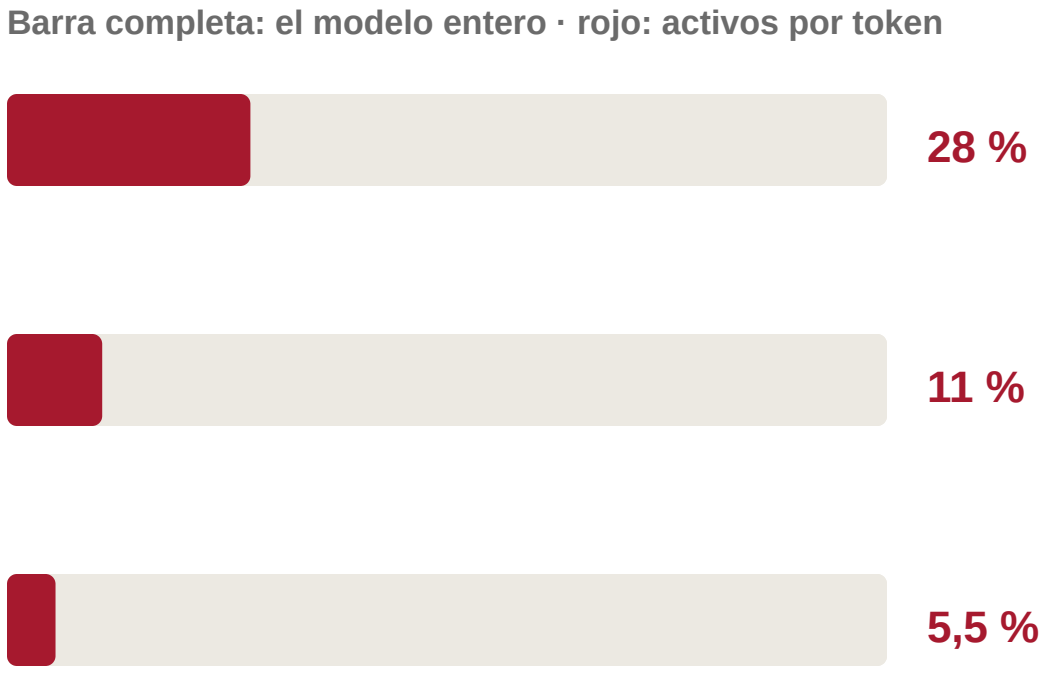
47 B total · 13 B activos

Qwen3-30B-A3B

30,5 B total · 3,3 B activos

DeepSeek-V3

671 B total · 37 B activos



Toda la memoria se carga, pero solo se calcula una parte

Fuente: model cards de Mixtral-8x7B-v0.1, DeepSeek-V3 y Qwen3-30B-A3B (Hugging Face); arXiv:2401.04088; consulta 2026-11-05

Modelos abiertos frente a cerrados

Pesos descargables frente a acceso solo por API

Abierto

Pesos que descargas

1. Corre donde quieras
2. La licencia varía

Cerrado

Acceso solo por API

1. Sin mantenimiento
2. Dependes del proveedor

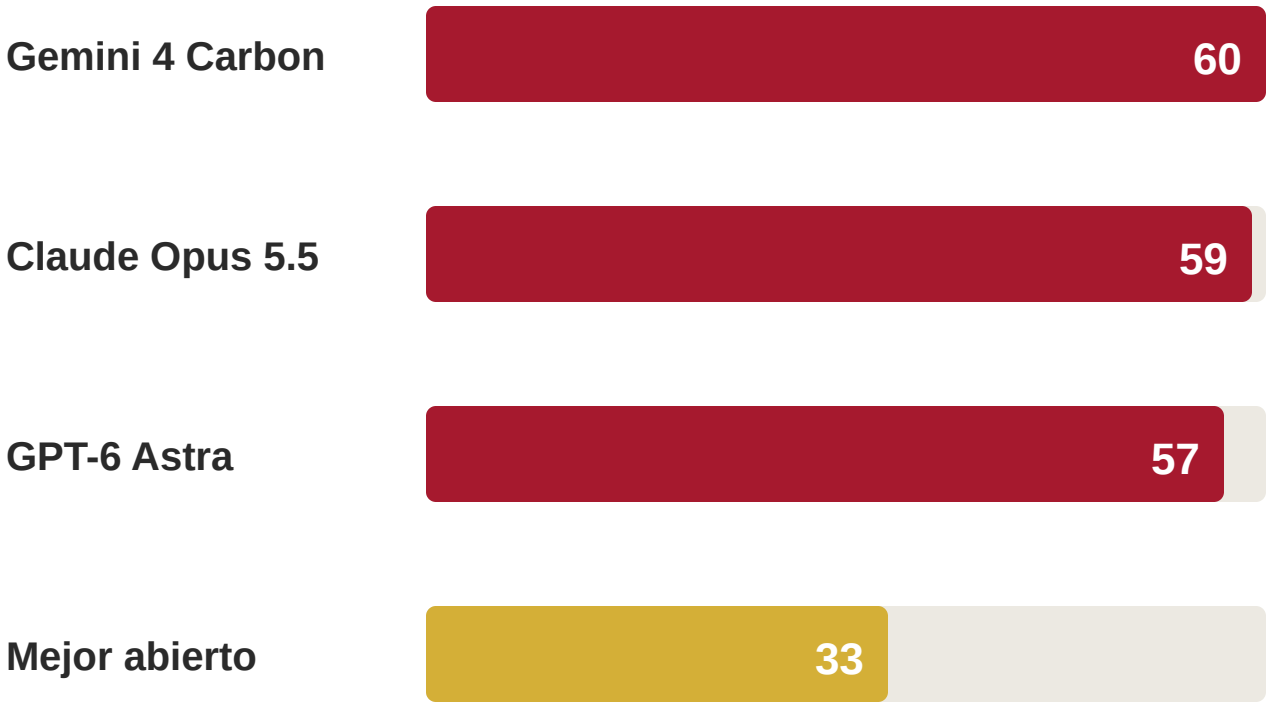
Abierto no siempre es libre: lee la licencia

API = interfaz de programación

Fuente: model cards en Hugging Face (Mixtral y Qwen3: Apache 2.0; DeepSeek-V3: licencia propia); consulta 2026-11-05

Dos rankings para comparar modelos

Índice de Artificial Analysis, consulta del 5 de noviembre de 2026



Arena
Votos humanos a ciegas

Artificial Analysis
Pruebas, precio, velocidad

El orden cambia cada pocas semanas: consulta en vivo

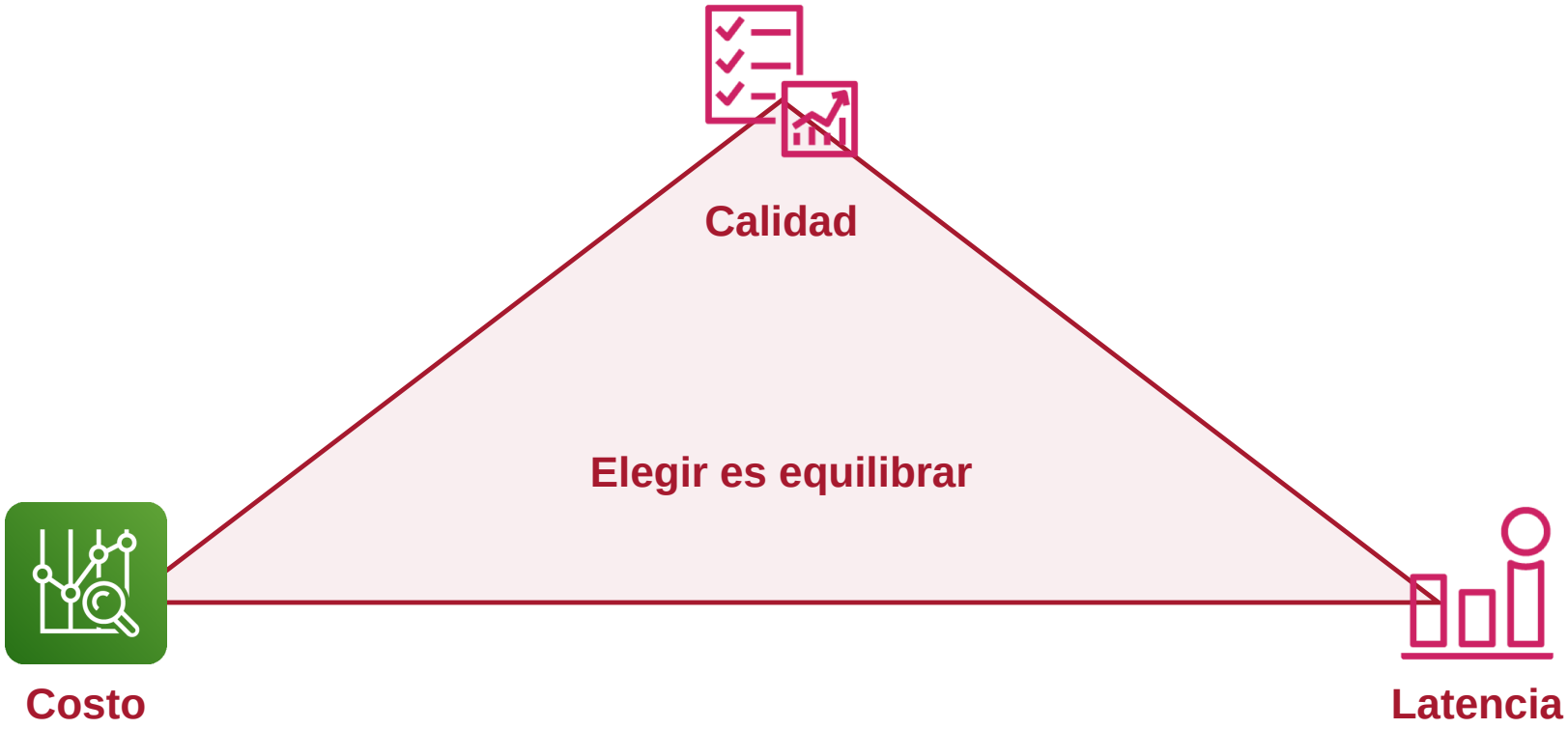
GPT = transformador generativo preentrenado

Fuente: artificialanalysis.ai (leaderboards y open weights, consulta 2026-11-05, la página no indica fecha de corte); arena.ai (no verificado en vivo)

PREGUNTA GUÍA

¿Necesitas siempre el modelo más potente?

Tres ejes que compiten entre sí



Rara vez hace falta el máximo en los tres ejes

Fuente: criterio docente; tarifas de Amazon Bedrock del archivo local (consulta 2026-11-05)

PREGUNTA GUÍA

Tarea fácil: modelo pequeño

Cada tarea se asigna al modelo más barato que la resuelve bien

Clasificar | **Extraer datos** | **Dar formato** → **Modelo pequeño**
Barato y rápido

Planificar | **Razonar** | **Auditar** → **Modelo grande**
Caro y más lento

No uses un camión para llevar una carta

Fuente: Ong et al. (2024), RouteLLM, arXiv:2406.18665; tarifas de la diapositiva 53

Planificar con un modelo, ejecutar con otro

El grande decide y verifica; el pequeño trabaja



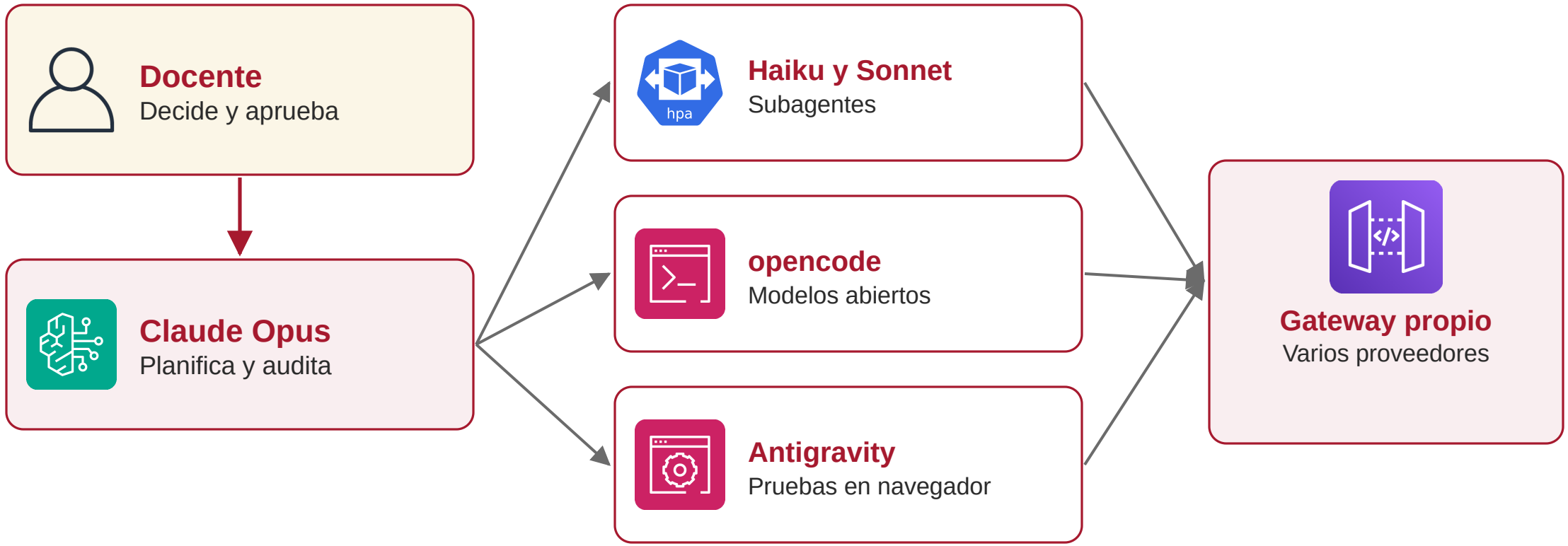
Un «verificado» del delegado no basta

Fuente: práctica del proyecto ASUR (laboratorio del docente)

CASO REAL

ASUR: el laboratorio del docente

Quién decide, quién planifica, quién ejecuta



Cada modelo hace lo que mejor le sale al menor costo

ASUR = laboratorio del docente

Fuente: proyecto ASUR, laboratorio del docente para su docencia (documentación interna)

CASO REAL

Cómo se entrena el contexto

Cinco piezas que mantienen alineado al modelo

1	**CLAUDE.md y memoria**	Reglas y lecciones que persisten
2	**Skills**	Paquetes de instrucciones reutilizables
3	**Subagentes**	Tareas en paralelo, contexto propio
4	**Objetivos verificables**	Criterio claro de terminado
5	**Auditoría**	Evidencia propia, no solo «verificado»

El contexto se construye, no se improvisa

CLAUDE = archivo CLAUDE.md con reglas y contexto del proyecto

Fuente: práctica del proyecto ASUR; documentación de Claude Code (docs.claude.com), consulta 2026-11-05

Vibe coding: programar describiendo la intención

Rápido, pero con riesgos que hay que compensar

Riesgos	Cómo se compensa
Código sin entender	Auditoría con evidencia
Fallos silenciosos	Controles automáticos
Seguridad por descuido	Aprobación humana

La velocidad sirve si alguien verifica

Fuente: término de A. Karpathy (2025); práctica del proyecto ASUR

CASO REAL

Así se rehicieron estas diapositivas

Del estándar a la tercera pasada



1 · Estándar

El grande lo define



2 · Delegar

Una semana por modelo



3 · Revisar

Hojas de contacto



4 · Repetir

Segunda y tercera pasada

Controles automáticos



Desbordes



Siglas



Tamaño de texto

El humano aprueba; las máquinas verifican

Fuente: historial de trabajo de la Unidad 3 (proyecto ASUR)

CIERRE DE SESIÓN

Conclusiones y conexión con la Semana 16

Cuatro ideas de la parte 5



1. IA local
Privacidad y práctica



2. MoE
Total frente a activo



3. Modelo justo
Costo, latencia, calidad



4. Delegar
Planificar y auditar

Próxima semana: IA III

IA = inteligencia artificial · MoE = mezcla de expertos · III = tercera (número romano)

Fuente: síntesis pedagógica de la parte 5 de la Semana 15